Quaderno AI

Concetti, strumenti e percorsi pratici per lo sviluppo software con AI

Andrea Merlin

Numero 1 - Novembre 2026

Indice

Capitolo 5 **1**

Migrare a .NET da altri linguaggi e piattaforme 1
5.1 Perché migrare e come valutare il progetto 3
5.2 Mappare linguaggi, runtime e strumenti 5
5.3 Pianificare una migrazione controllata 9
5.4 Architettura e organizzazione del codice 12
5.5 Richieste HTTP e pipeline 16
5.6 Configurazione, stato e dipendenze 19
5.7 Dati e integrazione con i sistemi esistenti 23
5.8 Sicurezza e identità 27
5.9 Testing e qualità 30
5.10 Prestazioni e comportamento del runtime 34
5.11 Container, CI/CD e produzione 36
5.12 Caso pratico: migrare un servizio catalogo 39
5.13 Approfondire la mappa dai principali ecosistemi 45
5.14 Prove di migrazione dei dati: runbook e controlli 49
5.15 Diagnosi operativa durante la migrazione 52
5.16 Verifica finale: criteri di successo 54
5.17 Checklist, glossario e anti-pattern 54
5.18 Appendice: piano di migrazione riutilizzabile 57

Capitolo 5

Migrare a .NET da altri linguaggi e piattaforme

Abstract. Migrare un'applicazione non significa riscrivere ogni riga cambiando sintassi. Significa trasferire in un nuovo ecosistema un insieme di comportamenti, dati, contratti, vincoli operativi e aspettative degli utenti, riducendo il rischio di interrompere il servizio. Questo capitolo guida chi conosce già PHP, Python, JavaScript/Node.js, Java o un altro stack attraverso il passaggio a .NET moderno e ASP.NET Core. Partiremo dall'assessment del sistema esistente e confronteremo le strategie di migrazione; poi costruiremo una mappa dei concetti, esploreremo hosting, pipeline HTTP, Dependency Injection, dati, sicurezza, test, prestazioni e deployment. Un caso pratico mostrerà come migrare un servizio per passi, mantenendo compatibili i client e predisponendo rollback e osservabilità. L'obiettivo non è dichiarare un framework universalmente migliore, ma fornire un metodo verificabile per scegliere cosa trasferire, cosa adattare e cosa lasciare temporaneamente nel sistema originale.

Cosa imparerai in questo capitolo

Al termine saprai:

- distinguere una migrazione tecnologica da una riscrittura funzionale e raccogliere le informazioni necessarie prima di stimare il lavoro;
- confrontare PHP, Python, Node.js e Java con C# e .NET senza confondere analogie utili con equivalenze perfette;
- scegliere tra riscrittura completa, migrazione per moduli e sostituzione incrementale con il pattern *strangler*;
- creare e organizzare un servizio ASP.NET Core, capire il ruolo dell'host, di Kestrel e di `Program.cs`;
- trasferire route, middleware, validazione, error handling, configurazione e stato;
- pianificare schema, migrazioni e dati con Entity Framework Core mantenendo le compatibilità necessarie;
- migrare autenticazione, autorizzazioni e segreti senza abbassare il livello di protezione;

- costruire test che descrivano il comportamento attuale e verificare la parità funzionale;
- misurare prestazioni, integrare CI/CD, distribuire in container e preparare rollout e rollback;
- usare checklist e criteri di uscita per dimostrare che la migrazione è pronta.

Prerequisiti

- Conoscenza di base di almeno un linguaggio e framework di sviluppo.
- Familiarità con HTTP, API, database relazionali, test automatici e controllo versione.
- Un SDK .NET 10 installato per eseguire gli esempi. Non è richiesta esperienza precedente con C# o ASP.NET Core.

Convenzioni

- I comandi sono mostrati in forma multipiattaforma con `dotnet`; i frammenti di shell possono richiedere piccoli adattamenti in PowerShell.
- Il riferimento è .NET 10, salvo indicazione diversa. Le API e i template possono variare tra versioni: verificare sempre il target dichiarato nel `.csproj`.
- Gli esempi sono didattici: prima di usarli in produzione occorre adattare autenticazione, segreti, storage, log e policy operative.
- "Sistema sorgente" indica l'applicazione da cui si migra; "sistema target" indica quella realizzata con .NET. "Cutover" indica il passaggio del traffico al target.

Indice concettuale

1. Perché migrare e come valutare il progetto
2. Mappare linguaggi, runtime e strumenti
3. Pianificare una migrazione controllata
4. Architettura e organizzazione del codice
5. Richieste HTTP e pipeline
6. Configurazione, stato e dipendenze
7. Dati e integrazione con i sistemi esistenti
8. Sicurezza e identità
9. Testing e qualità
10. Prestazioni e comportamento del runtime
11. Container, CI/CD e produzione
12. Caso pratico: migrare un servizio catalogo
13. Approfondire la mappa dai principali ecosistemi
14. Prove di migrazione dei dati: runbook e controlli
15. Diagnosi operativa durante la migrazione
16. Verifica finale, criteri di successo e fonti ufficiali
17. Checklist, glossario e anti-pattern

18. Appendice: piano di migrazione riutilizzabile

5.1 Perché migrare e come valutare il progetto

Una migrazione viene spesso proposta come risposta a un problema percepito: un framework è fuori supporto, il team non riesce più a trovare competenze, il costo di manutenzione cresce o il servizio deve entrare in un ambiente standardizzato su .NET. La proposta tecnica, però, non dimostra da sola che la migrazione sia vantaggiosa. Prima di scegliere la destinazione bisogna comprendere il sistema che si intende sostituire, il valore che offre e i vincoli che non sono visibili osservando soltanto i file sorgente.

5.1.1 Migrazione non è traduzione

Un'applicazione è più del suo codice. Comprende almeno:

- comportamenti osservabili, inclusi casi limite e comportamenti accidentali su cui i client hanno finito per fare affidamento;
- contratti HTTP, formati, codici di stato, eventi, code, file e schemi dati;
- identità, ruoli, permessi, credenziali e regole di conservazione;
- operazioni pianificate, procedure manuali e strumenti di supporto;
- caratteristiche operative: latenza, disponibilità, throughput, costo e procedure di ripristino.

Trasferire ogni funzione una a una può perpetuare difetti e vincoli non più necessari. Cambiare tutto insieme, d'altra parte, rende difficile capire se un malfunzionamento dipende dalla nuova piattaforma o da una modifica funzionale. In una migrazione tecnologica ben controllata si definisce prima quali comportamenti devono restare invariati. Le nuove funzionalità e le correzioni di dominio possono essere introdotte separatamente, oppure identificate esplicitamente come variazioni di contratto.

Una distinzione utile è tra **parità funzionale** e **parità operativa**. La prima riguarda ciò che utenti e client possono fare; la seconda riguarda come il servizio si comporta sotto carico, come viene distribuito e osservato, quali procedure servono per ripristinarlo. Un endpoint che restituisce lo stesso JSON ma non rispetta più il limite di latenza del client è funzionalmente simile, non una sostituzione riuscita.

5.1.2 Motivazioni e controindicazioni

Tra le motivazioni ragionevoli ci sono il termine del supporto del runtime, la difficoltà di aggiornare dipendenze critiche, i requisiti di sicurezza o conformità, l'esigenza di unificare toolchain e infrastruttura, l'integrazione con librerie .NET già adottate e la necessità di sostenere un carico o una modalità di deployment diversi.

Non sono sufficienti, da sole, frasi come “C# è più veloce”, “il linguaggio è vecchio” o “vogliamo un’architettura moderna”. Un’applicazione stabile, ben testata e con costo operativo contenuto può essere più sicura da aggiornare sullo stack corrente. Una riscrittura consuma capacità di sviluppo, crea un periodo di doppia manutenzione e può introdurre regressioni. Il confronto deve includere il costo totale: sviluppo, licenze, hosting, supporto, formazione, incidenti e dismissione del vecchio sistema.

La domanda corretta non è «.NET è migliore?», ma «quali requisiti del nostro sistema soddisfa meglio il target, e con quali costi e rischi misurabili?». Un prototipo può validare un’integrazione o una libreria; non prova automaticamente che una migrazione completa sia conveniente.

5.1.3 Inventario tecnico e funzionale

L’assessment inizia da un inventario verificabile. Non basta contare controller e righe di codice. Raccogliere, dove disponibili:

1. **Superfici funzionali:** route, metodi, parametri, payload, errori, pagine e client che le usano.
2. **Processi:** job schedulati, consumer di code, import/export, webhook, strumenti amministrativi e script manuali.
3. **Dati:** database, tabelle, volumi, crescita, vincoli, indici, backup, retention e migrazioni pregresse.
4. **Dipendenze:** runtime, framework, pacchetti, servizi cloud, SDK, sistemi di identità e componenti nativi.
5. **Integrazioni:** API esterne, code, SMTP, storage di file, pagamenti, sistemi legacy e formati di scambio.
6. **Operazioni:** pipeline, deployment, secret store, logging, monitoraggio, alert, runbook, RTO e RPO.
7. **Utilizzo:** frequenza delle route, profili di carico, picchi, client attivi, operazioni costose e stagionalità.
8. **Vincoli:** finestre di fermo, requisiti legali, localizzazione dei dati, compatibilità, competenze e budget.

Accoppiare l’inventario statico con osservazioni runtime. Log, tracing e metriche rivelano percorsi davvero usati e dipendenze effettive; repository e documentazione rivelano intenzioni e configurazioni. Nessuna delle due fonti è completa. Un route presente nel codice può essere inutilizzato, ma non va rimosso senza verifica con i proprietari dei client.

5.1.4 Requisiti non funzionali e baseline

Per valutare il target servono misure di partenza. Registrare almeno:

- latenza per operazioni rappresentative, includendo percentili come p50, p95 e p99;
- richieste al secondo, concorrenza, code e frequenza degli errori;

- consumo di CPU, memoria, storage, connessioni e risorse di database;
- tempi di avvio, deploy, ripristino e build;
- disponibilità, incidenti, obiettivi RTO/RPO e carico operativo del team.

Misurare le stesse operazioni con input e ambiente confrontabili. Una media globale nasconde le route lente e la variabilità; un benchmark su una singola macchina non rappresenta una produzione con rete, database e cache. Annotare configurazione, versione, dimensione del dataset e data della misura. Senza una baseline il team non può distinguere un miglioramento reale da una differenza di ambiente.

Definire criteri d'accettazione prima dell'implementazione. Per esempio: tutti i flussi critici hanno test di caratterizzazione; il target rispetta la compatibilità API; p95 non supera il budget concordato; il rollback è stato provato; nessun segreto è contenuto nell'immagine. I numeri devono provenire dal servizio e dai suoi utenti, non da soglie arbitrarie.

5.1.5 Registro delle decisioni e dei rischi

Per ogni funzione si può registrare proprietario, frequenza, criticità, dipendenze e strategia proposta. Un registro decisionale breve dovrebbe descrivere la decisione, le alternative considerate, le evidenze e le condizioni che la farebbero rivedere. Esempio: mantenere inizialmente il database invariato perché più servizi condividono lo schema; riconsiderare la separazione solo dopo aver stabilizzato API e ownership.

Valutare probabilità e impatto dei rischi: dati non interpretabili allo stesso modo, dipendenza non portabile, identità non migrabili, traffico non riproducibile, doppia scrittura incoerente, indisponibilità durante il cutover o mancanza di rollback. Ogni rischio importante deve avere una mitigazione e un responsabile. Se manca un'informazione critica, trasformarla in un'attività di discovery con un risultato atteso, non in una supposizione nascosta nel piano.

Risultato dell'assessment: una mappa del sistema, una baseline, un insieme di contratti da preservare, una lista di incertezze e una raccomandazione motivata tra migrare, modernizzare sullo stack esistente o rimandare. Questo pacchetto è la base della pianificazione, non un documento da archiviare senza seguito.

5.2 Mappare linguaggi, runtime e strumenti

Chi arriva in .NET porta con sé un modello mentale valido per il proprio ecosistema. Riutilizzarlo rende più rapido l'apprendimento, a patto di non supporre che due strumenti con nomi o scopi simili abbiano lo stesso lifecycle, la stessa semantica o gli stessi limiti. In questa sezione costruiamo una mappa e una piccola applicazione che useremo nei capitoli successivi.

5.2.1 Da .NET Core a .NET moderno

".NET Core" è il nome della linea open source multipiattaforma fino a .NET Core 3.1. Da .NET 5 il prodotto si chiama **.NET**: .NET 6, 7, 8, 9 e 10 sono versioni della piattaforma unificata. **ASP.NET Core** è il framework web moderno della piattaforma e ha mantenuto "Core" nel proprio nome. Il vecchio **.NET Framework** è un prodotto distinto, principalmente legato a Windows; non è il target di questo capitolo.

Per un nuovo servizio, fissare una versione supportata e dichiararla esplicitamente. .NET 10 è il riferimento degli esempi. Non scegliere il target perché il computer dello sviluppatore lo ha installato per caso: SDK, target framework, pipeline CI e runtime di produzione devono essere coordinati. Per la policy di supporto e le patch correnti fare riferimento alla pagina ufficiale del ciclo di vita .NET.

5.2.2 Concetti e strumenti a confronto

- **PHP:** linguaggio PHP; runtime PHP e Composer; progetto `composer.json`; avvio dipendente dal framework o dal worker; framework come Laravel e Symfony; pacchetti da Packagist.
- **Python:** linguaggio Python; runtime e pip/Poetry; progetto `pyproject.toml`; avvio tramite modulo, ASGI o WSGI; framework come Django, Flask e FastAPI; pacchetti da PyPI.
- **Node.js:** JavaScript o TypeScript; Node.js con npm/pnpm; progetto `package.json`; entry point Node; framework come Express e NestJS; pacchetti dal registry npm.
- **Java:** linguaggio Java; JDK con Maven/Gradle; progetto `pom.xml` o file Gradle; avvio da `main` o framework; framework come Spring Boot; pacchetti da Maven Central.
- **.NET:** linguaggio C#; SDK e runtime .NET; progetto `.csproj`; avvio da `Program.cs` e host; framework come ASP.NET Core; pacchetti da NuGet.

La tabella orienta, ma non mappa capacità uno-a-uno. Per esempio un provider del framework sorgente può combinare routing, dependency injection, sessione e configurazione; nel target queste responsabilità possono appartenere a servizi differenti. Prima di sostituirlo, scomporre quali comportamenti effettivamente offre.

5.2.3 SDK, runtime e CLI

L'**SDK** include compilatore, template e strumenti per sviluppare; il **runtime** esegue un'applicazione già pubblicata. Un server di build richiede l'SDK; un container di produzione può usare solo il runtime, oppure un artefatto self-contained a seconda della strategia. La distinzione influenza immagini, dimensione del deploy, patching e installazione.

La CLI standardizza molte operazioni:

```
dotnet --info
dotnet new webapi -n Catalogo.Api
cd Catalogo.Api
dotnet restore
dotnet build
dotnet test
dotnet run
dotnet publish -c Release -o publish
```

`dotnet restore` risolve i pacchetti; `build` compila; `test` esegue i test; `run` avvia il progetto; `publish` produce l'output destinato alla distribuzione. Non sono sinonimi: per esempio una build riuscita non garantisce che l'applicazione sia configurata correttamente in produzione.

Per pinning riproducibile, il repository può contenere `global.json` con la policy SDK usata dal team. Il file `.csproj` dichiara il target:

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

`TargetFramework` non è una lista di pacchetti. Le librerie esterne si aggiungono con `dotnet add package` e vengono registrate nel progetto; in una soluzione con più progetti si distingue il progetto API dai progetti di dominio e test.

5.2.4 Sintassi C# per chi conosce già un linguaggio

C# è staticamente tipizzato e compilato. Il compilatore intercetta categorie di errori prima dell'avvio, ma il sistema dei tipi non sostituisce validazione e test. `string?` indica che un valore può essere null quando nullable reference types sono attivi; `string` esprime l'intenzione opposta. I dati ricevuti da HTTP sono comunque input non attendibile e vanno validati.

Un record può descrivere un DTO immutabile:

```
public sealed record CreateProductRequest(string Name, decimal Price);
```

La definizione non valida automaticamente `Name` o `Price`: occorrono regole esplicite. Un metodo asincrono restituisce normalmente `Task<T>` o `ValueTask<T>` e usa `await`; l'asincronia è essenziale per non bloccare thread mentre si aspetta I/O, ma non rende automaticamente veloce una query.

```
public async Task<Product?> FindAsync(
    int id, CancellationToken cancellationToken)
{
```

```
    return await db.Products
        .SingleOrDefaultAsync(p => p.Id == id, cancellationToken);
}
```

Il codice non dovrebbe nascondere le eccezioni con `catch { return null; }`: un errore database non equivale a un prodotto assente. Gli errori devono essere tradotti in modo coerente al confine applicativo e registrati con contesto sufficiente, senza esporre dettagli interni al client.

5.2.5 Lifecycle e hosting

Molti framework sorgente offrono un processo per richiesta, un worker persistente, oppure più processi con un server applicativo davanti. ASP.NET Core usa un **host** che costruisce configurazione, logging, dependency injection e lifecycle dell'applicazione. Kestrel è il server HTTP integrato; può ricevere direttamente traffico o lavorare dietro un reverse proxy, in base all'ambiente.

Nel modello moderno `Program.cs` configura servizi e pipeline e avvia l'host. La parte essenziale può essere molto breve:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();
app.MapGet("/health", () => Results.Ok(new { status = "ok" }));
app.Run();
```

`AddEndpointsApiExplorer()` da solo registra metadati, ma non genera né espone un documento OpenAPI: serve insieme a un generatore. Per esempio, in .NET 10, aggiungendo il pacchetto `Microsoft.AspNetCore.OpenApi`, si può usare `AddOpenApi()` con `MapOpenApi()`; qui non è necessario e viene omesso.

La brevità non implica assenza di struttura: route, servizi, opzioni e componenti possono essere estratti in metodi o moduli. Il punto d'ingresso non è il luogo in cui inserire tutta la logica di dominio; è il punto in cui si compone l'applicazione.

5.2.6 Pacchetti e dipendenze

NuGet è il sistema di pacchetti .NET. Ogni dipendenza aggiunta aumenta superficie di aggiornamento, licenze e rischio di supply chain. Prima di portare un pacchetto, verificare compatibilità con il target, manutenzione, advisory, licenza, dipendenze transitive e necessità effettiva. Un equivalente con nome simile può avere una filosofia o un lifecycle differenti.

Per build riproducibili, versionare il file di lock quando il workflow lo usa, mantenere gli aggiornamenti automatizzati e non includere credenziali del registry nel progetto. Configurazioni di feed e token devono stare in ambienti o secret store protetti. La pipeline dovrebbe ripristinare, compilare e testare da una copia pulita del repository.

5.2.7 Un primo servizio e le verifiche di ambiente

Creare un progetto è solo la prima verifica. Prima di migrare logica applicativa, accertarsi che il team possa:

1. installare o selezionare la versione SDK prevista;
2. ripristinare i pacchetti dietro le restrizioni di rete del progetto;
3. compilare e testare dalla CLI e in CI;
4. avviare l'applicazione con la configurazione locale;
5. produrre un artefatto pubblicabile e avviarlo in un ambiente simile alla produzione.

Registrare i comandi funzionanti e le versioni nel README del progetto. Un "funziona sul mio IDE" non è ancora una toolchain condivisa.

5.2.8 La mappa deve essere verificata sul caso reale

Per ogni concetto del sistema sorgente, compilare una scheda con tre colonne: responsabilità attuale, candidato .NET, differenza da provare. Per esempio: un hook di richiesta in un framework Python può essere middleware, filtro o endpoint filter; la scelta dipende dal fatto che debba avvolgere l'intera richiesta, agire solo su una route o validare parametri. Questa verifica evita migrazioni basate sulla sola somiglianza dei nomi.

5.3 Pianificare una migrazione controllata

Scelta la direzione, bisogna decidere come raggiungerla. La strategia influenza tempi, costi, possibilità di rollback, coordinamento tra team e probabilità di mantenere il servizio disponibile. Non esiste una tecnica universalmente corretta: la decisione dipende dai confini funzionali, dalle dipendenze condivise, dalle capacità di test e dai vincoli del business.

5.3.1 Tre strategie, non due

Riscrittura completa (big bang). Il sistema viene ricostruito in un ramo o ambiente separato e sostituito in un'unica data. È semplice da spiegare, ma raramente semplice da eseguire: per molto tempo due implementazioni divergono e il team scopre tardi differenze nei dati e nei comportamenti. Ha senso solo con superficie contenuta, confini chiari, test affidabili, breve durata e un cutover reversibile. Un lungo congelamento funzionale può ridurre la divergenza, ma comporta costi di business.

Migrazione per componenti. Si selezionano servizi o moduli autonomi e li si migra in sequenza. Riduce il perimetro di ogni rilascio, a condizione che i confini siano reali. Un modulo solo nominalmente separato ma accoppiato a tabelle condivise, sessioni o chiamate interne non è necessariamente una buona unità di migrazione.

Sostituzione incrementale (*strangler*). Un router, gateway o livello di ingresso dirige alcune capacità verso il sistema nuovo e mantiene le restanti sul sistema originale. Le funzionalità vengono spostate progressivamente; alla fine si rimuove il vecchio sistema. Consente feedback reale e rollback selettivo, ma richiede di gestire coesistenza, identità, osservabilità e contratti tra i due lati. Non è sinonimo di riscrittura in microservizi.

Strategia	Rischio concentrato	Vantaggio	Condizione che la rende praticabile
Big bang	Cutover e regressioni aggregate	Confine di transizione semplice	Sistema piccolo e test completo
Per componenti	Integrazione tra moduli	Progressi e rollback per unità	Confini e ownership chiari
Strangler	Coesistenza e routing	Migrazione graduale in produzione	Ingresso instradabile e contratti stabili

Una strategia ibrida è frequente: si crea un nuovo edge applicativo, si migrano prima funzionalità isolate, si sostituiscono poi i flussi con dipendenze più forti e si pianifica una finestra di cutover per elementi non separabili.

5.3.2 Scegliere la prima capacità

La prima funzionalità migrata non deve essere né un semplice endpoint senza valore, né il flusso più critico dell'azienda. È preferibile una capacità rappresentativa ma reversibile: possiede un contratto comprensibile, ha dipendenze sotto controllo e permette di esercitare build, deploy, logging, test e routing.

Valutare ogni candidato considerando valore, rischio, dipendenze, volume di traffico e facilità di verifica. Un indice sintetico può aiutare a confrontare opzioni, ma non deve sostituire la discussione: un punteggio numerico non elimina l'incertezza. Il team dovrebbe poter spiegare perché una capacità è pronta e quali evidenze mancano.

Prima di spostare il traffico occorre sapere come il router identifica la funzionalità: prefisso di route, host, tenant, feature flag o regola di gateway. Se la decisione dipende da un flag, deve essere possibile disabilitarlo rapidamente senza ricompilare o perdere tracciabilità delle modifiche. Un flag che resta indefinitamente nel codice non è un piano di migrazione.

5.3.3 Compatibilità durante la coesistenza

Due sistemi attivi contemporaneamente possono leggere e scrivere lo stesso dato. Le letture sono spesso più facili da spostare; le scritture richiedono un

proprietario chiaro. Scrivere su due database indipendenti in sequenza non crea una transazione atomica: una scrittura può riuscire e l'altra fallire. La doppia scrittura non va introdotta come soluzione rapida senza un protocollo di riconciliazione, idempotenza e gestione dei conflitti.

Le alternative dipendono dal dominio:

- un solo sistema possiede temporaneamente la scrittura; l'altro legge dati replicati;
- si trasferisce la proprietà per entità o gruppo, con un registro di ownership;
- si usa un outbox transazionale per pubblicare cambiamenti dopo il commit;
- si effettua una migrazione con snapshot iniziale e sincronizzazione incrementale.

Ogni opzione ha costi e failure mode. L'outbox evita di perdere l'intenzione di pubblicare un evento nella stessa transazione locale, ma non rende atomici database diversi e consumer. La consegna può essere ripetuta; i consumer devono essere idempotenti. In tutte le opzioni vanno definiti conflitti, retry, deduplicazione e procedura di riconciliazione.

Anche i contratti devono evolvere con compatibilità. Aggiungere un campo opzionale è spesso più sicuro che rinominare o cambiare il tipo di uno esistente; tuttavia il client potrebbe rifiutare campi sconosciuti o interpretare diversamente un default. Versionamento e compatibilità vanno provati contro i client reali o le loro specifiche.

5.3.4 Fasi con condizioni di uscita

Una sequenza utile è:

1. **Discovery:** inventario, baseline, dipendenze e rischi approvati.
2. **Foundation:** SDK/CI, hosting, log, configurazione, sicurezza e deploy vuoto verificati.
3. **Caratterizzazione:** contratti ed esempi rappresentativi fissati nei test.
4. **Vertical slice:** un flusso completo attraversa HTTP, dominio, persistenza, test e produzione non critica.
5. **Migrazione per capacità:** ogni unità ha criteri d'accettazione, osservazione e rollback.
6. **Cutover:** traffico spostato con una finestra e responsabili espliciti.
7. **Stabilizzazione e rimozione:** dati riconciliati, dipendenze eliminate, vecchio sistema dismesso e costi ridotti.

Le fasi possono sovrapporsi, ma le condizioni d'uscita devono essere esplicite. Per esempio, la foundation non è completata se esiste soltanto un Dockerfile: bisogna dimostrare che l'immagine gira nell'ambiente previsto, riceve configurazione esterna, espone health check e termina correttamente.

5.3.5 Cutover e rollback

Il cutover va trattato come un'operazione controllata, non come una modifica improvvisata al load balancer. Il piano include orario, responsabile della decisione, metriche da osservare, soglie d'arresto, canali di comunicazione, ownership dei dati e comandi di ritorno.

Un rollback tecnico non annulla automaticamente le scritture già fatte dal nuovo sistema. Se lo schema o il formato dati non sono leggibili dal vecchio sistema, tornare indietro può richiedere restore, replay o riconciliazione. Applicare migrazioni database compatibili in più passi: espandere lo schema, distribuire codice che supporta entrambi i formati, migrare i dati, poi rimuovere gli elementi obsoleti solo quando il rollback non li richiede più.

Provare il rollback prima del cutover. Registrare durata, autorizzazioni necessarie, effetti sui dati e segnali che confermano che il traffico è tornato sul vecchio percorso. Un piano non provato è un'ipotesi.

5.3.6 Organizzazione e dipendenze

La migrazione richiede decisioni condivise tra sviluppo, piattaforma, sicurezza, database, supporto e product owner. Stabilire chi approva un contratto, chi controlla le metriche e chi può fermare il rollout. Esplicitare le dipendenze tra attività, evitando che più team cambino lo stesso schema o endpoint senza coordinamento.

Il backlog deve distinguere lavoro di migrazione, correzioni necessarie per la parità e nuove funzionalità. Questa separazione rende visibile il costo e impedisce che una funzionalità aggiunta a metà diventi la ragione per rinviare la dismissione. Ogni milestone dovrebbe produrre un incremento funzionante e una decisione informata sul passo successivo.

5.4 Architettura e organizzazione del codice

L'architettura di destinazione non dovrebbe essere una copia dei package del framework sorgente né una collezione di pattern scelti per moda. Il suo compito è rendere espliciti i confini: dove termina HTTP, dove risiede la logica di dominio, chi possiede i dati e come si integrano le dipendenze esterne.

5.4.1 Dal framework sorgente alle responsabilità ASP.NET Core

Laravel, Django, Express/NestJS e Spring Boot offrono combinazioni diverse di routing, middleware, controller, container, configurazione e persistenza. Durante la migrazione, decomporre il comportamento:

Responsabilità sorgente	Possibile destinazione ASP.NET Core
Route/controller	Minimal API o controller MVC
Hook globale per richiesta	Middleware
Hook limitato a endpoint	Endpoint filter o filtro MVC
Container dei servizi	`IServiceCollection` e provider DI
Configurazione ambientale	Configuration providers e Options
Worker/job	`BackgroundService`, worker o scheduler esterno
Validazione input	Validazione esplicita, filtri o libreria adottata
Template/server rendering	Razor Pages, MVC o frontend separato

La tabella indica candidati, non una traduzione automatica. Un middleware ASP.NET Core avvolge una richiesta e può decidere se proseguire; un filtro MVC opera nel ciclo MVC; un endpoint filter si applica ai delegate degli endpoint. Scegliere il livello in base alla responsabilità evita middleware globali che contengono eccezioni specifiche per una sola route.

5.4.2 Minimal API o controller?

Le Minimal API sono un modo conciso per definire endpoint e si adattano bene a servizi piccoli, API focalizzate e team che vogliono comporre route per feature. Non sono un framework "senza architettura": dipendenze, validazione, errori, mapping e test devono comunque essere organizzati.

I controller MVC offrono convenzioni e struttura familiare per API articolate, applicazioni con molti endpoint simili, filtri MVC e team che beneficiano di classi e attributi. Non obbligano a usare tutte le funzionalità MVC. La scelta non va basata sul numero di righe iniziali: valutare complessità delle regole trasversali, standardizzazione, familiarità del team e necessità di organizzare i contratti.

Esempio Minimal API con gruppo:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<IProductService, ProductService>();

var app = builder.Build();
var products = app.MapGroup("/api/products");

products.MapGet("/{id:int}", async (
    int id, IProductService service, CancellationToken ct) =>
{
    var product = await service.FindAsync(id, ct);
    return product is null ? Results.NotFound() : Results.Ok(product);
});
```

```
app.Run();
```

Il delegate riceve parametri che ASP.NET Core associa a route, servizi o body secondo regole di binding. Per mantenere leggibile il punto di composizione, spostare la definizione in un metodo `MapProductEndpoints` o in un modulo quando le route crescono. Non trasformare subito ogni gruppo in un'astrazione generica: estrarre quando si chiarisce una responsabilità riutilizzabile.

5.4.3 Confini applicativi e dipendenze

Una struttura minima può separare:

- **API:** protocollo HTTP, autenticazione, mapping request/response;
- **Application:** casi d'uso e coordinamento;
- **Domain:** regole e invarianti di business;
- **Infrastructure:** database e integrazioni esterne.

Questi nomi non sono obbligatori. In un servizio piccolo, più progetti possono aggiungere overhead senza proteggere alcun confine. In un sistema con dominio complesso e più adapter, la separazione riduce l'accoppiamento. La direzione desiderata è che le regole del dominio non dipendano dal server web o dal provider database; i dettagli infrastrutturali implementano contratti necessari ai casi d'uso.

Un layout possibile per feature:

```
src/
  Catalogo.Api/
    Program.cs
    Features/
      Products/
        ProductEndpoints.cs
        CreateProductRequest.cs
  Catalogo.Application/
  Catalogo.Domain/
  Catalogo.Infrastructure/
tests/
  Catalogo.UnitTests/
  Catalogo.IntegrationTests/
```

Questo è uno schema, non un requisito. Una cartella `Features/Products` può contenere endpoint e request specifiche, mentre servizi applicativi e dominio vivono in progetti distinti. Evitare cartelle generiche `Helpers`, `Common` o `Utils` come destinazione per qualsiasi codice non collocato: diventano rapidamente dipendenze senza ownership.

5.4.4 Dominio, DTO e mapping

Non esporre direttamente l'entità database come contratto pubblico per comodità. Una modifica interna allo schema potrebbe cambiare involontariamente l'API; inoltre l'entità può contenere campi che il client non deve vedere. Usare DTO di richiesta e risposta quando esiste un confine reale, definendo mapping espliciti per campi, null e valori predefiniti.

```
public sealed record ProductResponse(
    int Id, string Name, decimal Price, string Currency);
```

La logica di conversione deve essere testata. Un mapping automatico può essere utile, ma non deve nascondere conversioni lossy, date senza timezone o proprietà omesse. Per ogni contratto documentare proprietà obbligatorie, formato, unità, precisione e compatibilità.

5.4.5 Hosting e Kestrel

`WebApplication.CreateBuilder(args)` compone configuration, logging, servizi e ambiente. L'app risultante può girare in foreground, in container o come servizio gestito. Kestrel gestisce HTTP; in produzione spesso c'è un reverse proxy o un ingress davanti. Questo proxy può terminare TLS, imporre limiti, fare routing e aggiungere header forwarded. L'app deve fidarsi solo dei proxy configurati, non di qualunque header fornito dal client.

Il binding delle porte e degli URL deve provenire dalla configurazione dell'ambiente. Non codificare `localhost` o porte di sviluppo nell'immagine. Verificare che il processo ascolti sull'interfaccia prevista, che i controlli health siano raggiungibili dal sistema di orchestrazione e che lo shutdown consenta di terminare richieste in corso.

5.4.6 Processi web, worker e job

Un'applicazione sorgente può eseguire job nello stesso processo HTTP. In .NET un `BackgroundService` è appropriato per lavoro persistente controllato dall'host, ma non è una coda durevole: se il processo si arresta, gli elementi in memoria possono andare persi. Per job che richiedono retry, scheduling distribuito o garanzie di consegna, usare una coda o un sistema di scheduling adeguato.

I worker devono gestire cancellazione e shutdown, limitare concorrenza, registrare gli errori e rendere idempotenti le operazioni ripetibili. Non avviare thread detached da un endpoint e non ignorare eccezioni in task non osservati. Il lifecycle del lavoro deve essere parte della strategia di deployment e scaling.

5.4.7 Strutturare per comprendere, non per imitare

Portare il modello mentale del vecchio framework è utile come punto di partenza, ma il target deve rispettare le sue convenzioni. Per ogni astrazione chiedersi:

quale responsabilità protegge? Quale test la rende verificabile? Quale cambiamento futuro diventa più semplice? Se non c'è una risposta, l'astrazione probabilmente anticipa un problema non ancora dimostrato.

Una migrazione ben strutturata è quella che permette di trovare rapidamente un endpoint, capire chi possiede una regola, sostituire una dipendenza esterna e testare il comportamento senza avviare l'intero sistema. Non è necessariamente quella con più progetti o livelli.

5.5 Richieste HTTP e pipeline

Un'API migra con successo solo se i suoi client osservano il contratto previsto. Cambiare framework spesso cambia dettagli che non erano scritti: codici di stato, forma degli errori, parsing di date, comportamento sui campi null, header, encoding e ordine delle operazioni. Questa sezione tratta la pipeline HTTP come un contratto da preservare e testare.

5.5.1 Ricostruire il contratto prima di spostare le route

Per ciascun endpoint registrare metodo e percorso, autenticazione, parametri, schema request/response, codici di stato, header, paginazione, ordinamento, idempotenza e limiti. Salvare esempi realistici, inclusi input invalidi e casi limite. OpenAPI aiuta a descrivere il contratto ma non sempre riflette il comportamento effettivo; confrontare specifica, implementazione e traffico anonimizzato.

Attenzione alle differenze di parsing:

- distinzione maiuscole/minuscole nei nomi JSON e nei valori;
- date locali, offset e precisione dei timestamp;
- separatori decimali e formati numerici;
- `null`, proprietà assente, stringa vuota e valore predefinito;
- encoding, Unicode normalizzato e limiti sulla dimensione del body;
- parametri duplicati, query string e codifica degli spazi.

Una modifica "più corretta" può essere breaking. Se si decide di correggere un comportamento, documentare la variazione e verificare la compatibilità dei consumer.

5.5.2 Middleware e ordine della pipeline

Il middleware è una sequenza. Un componente può modificare la richiesta prima del successivo, osservare la risposta dopo `await next()`, oppure terminare la pipeline. L'ordine influenza error handling, routing, autenticazione e autorizzazione.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddProblemDetails();
```

```
builder.Services.AddControllers();
builder.Services.AddAuthentication();
builder.Services.AddAuthorization();

var app = builder.Build();

app.UseExceptionHandler();
app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Per rendere operativo il gestore standard, registrare prima `AddProblemDetails`. In assenza di un handler personalizzato, il middleware restituisce una risposta Problem Details quando il servizio è disponibile. L'ordine reale dipende da servizi e hosting: seguire la documentazione della versione ASP.NET Core scelta. In generale il gestore eccezioni deve avvolgere i componenti che si vogliono gestire; autenticazione deve precedere autorizzazione; il routing deve aver stabilito l'endpoint prima che componenti dipendenti dall'endpoint lo usino. Non copiare meccanicamente un ordine trovato in un progetto diverso.

Un middleware per correlare le richieste deve propagare un identificatore sicuro e aggiungere contesto ai log, ma non deve fidarsi ciecamente di un valore arbitrario passato dall'esterno. Anche un header di correlazione può essere usato per iniettare caratteri o dati ingannevoli nei log; normalizzare e limitare la lunghezza.

5.5.3 Routing e binding

Minimal API e controller offrono convenzioni diverse per dichiarare route e binding. Una route esplicita rende visibile il contratto:

```
app.MapGet("/api/products/{id:int}", async (
    int id, IProductService service, CancellationToken ct) =>
{
    var item = await service.FindAsync(id, ct);
    return item is null
        ? Results.NotFound()
        : Results.Ok(item);
});
```

La constraint `:int` restringe il matching; non sostituisce una risposta coerente per input non valido. Per request complesse definire un tipo DTO anziché legare un'entità interna. Evitare parametri implicitamente ambigui e testare l'overload di route, slash finali, versioni e route non trovate.

La validazione deve avvenire al confine e includere vincoli di dominio pertinenti. Un controllo sintattico (campo obbligatorio, lunghezza massima) è diverso da una regola di business (prodotto attivo, prezzo modificabile). Restituire errori leggibili e stabili, senza stack trace o dettagli di database.

5.5.4 Errori prevedibili

Definire una tassonomia di errori: input non valido, risorsa inesistente, conflitto, non autenticato, non autorizzato, dipendenza non disponibile e errore inatteso. Mappare ogni categoria a status code e corpo consistente. `ProblemDetails` è un formato standard utile per risposte HTTP, ma i dettagli sensibili devono essere esclusi in produzione.

Le eccezioni non dovrebbero essere usate come controllo di flusso ordinario. La logica applicativa può restituire un risultato esplicito per conflitti attesi; il middleware centrale gestisce gli errori inattesi, registra l'eccezione e restituisce un identificatore di correlazione. Non restituire HTTP 200 con `{ "success": false }` se il contratto prevede status semantici; non trasformare ogni errore in 500 senza classificazione.

Un `catch` locale è appropriato se il componente può recuperare, aggiungere contesto utile o tradurre una specifica eccezione in un risultato noto. Altrimenti lasciar propagare al gestore centrale. Evitare log duplicati della stessa eccezione a ogni livello.

5.5.5 Filtri, middleware ed endpoint filter

La scelta del punto d'estensione dipende dall'ambito:

- **middleware:** cross-cutting a livello di pipeline, per log, header o gestione globale;
- **filtro MVC:** comportamento nel lifecycle dei controller, per endpoint MVC;
- **endpoint filter:** logica riutilizzabile per gruppi di Minimal API, inclusi controlli su argomenti;
- **codice applicativo:** regole specifiche del caso d'uso che devono essere testabili senza HTTP.

Non collocare regole di dominio in middleware solo perché è il primo hook conosciuto. Un middleware non dovrebbe conoscere il concetto di "ordine annullato" o "cliente premium".

5.5.6 Timeout, cancellazione e resilienza

Ogni chiamata remota può fallire o impiegare troppo tempo. Propagare `CancellationToken` dalle API alle query e ai client HTTP; applicare timeout espliciti e limiti per componente. Un timeout del client non garantisce che l'operazio-

ne remota sia stata annullata: l'esito può essere ambiguo, quindi le operazioni di scrittura devono considerare idempotency key o meccanismi di verifica.

Retry automatici sono adatti solo a errori transitori e operazioni sicure da ripetere. Non fare retry indiscriminati su ogni eccezione o su operazioni non idempotenti. Usare backoff con jitter, budget complessivo, circuit breaker se utile e telemetria che mostri i tentativi. Evitare retry annidati in più livelli: moltiplicano la pressione sul servizio già degradato.

5.5.7 CORS, rate limiting e documentazione

CORS è una policy applicata dai browser, non un'autenticazione. Consentire origini, metodi e header minimi; non combinare wildcard con credenziali. Il rate limiting protegge risorse e client equamente, ma una policy in memoria per istanza non produce limiti globali in un cluster. Scegliere chiave e storage coerenti con il deployment, proteggendo le chiavi da spoofing.

OpenAPI descrive endpoint e schemi e può generare client, ma la specifica deve essere parte della revisione. Mantenere esempi, errori, autenticazione e versionamento aggiornati. Diff automatizzati tra specifiche aiutano a individuare breaking change prima del rilascio.

5.5.8 Checklist della parità HTTP

Prima di spostare il traffico verificare: status code; schema e casing JSON; header; autenticazione; codici di errore; date e timezone; paginazione; ordinamento; CORS; timeout; payload grandi; input malformati; richieste duplicate; cancellazione e versionamento. Eseguire i test contro vecchio e nuovo sistema con gli stessi casi, confrontando gli output normalizzati e documentando ogni differenza intenzionale.

5.6 Configurazione, stato e dipendenze

I framework di partenza spesso rendono facili da condividere servizi, configurazione e stato di richiesta. In ASP.NET Core queste responsabilità sono esplicite: il container DI crea le dipendenze, i provider di configurazione aggregano valori e i servizi di cache o sessione gestiscono lo stato. Esplicitarle è un vantaggio solo se il lifecycle scelto è corretto.

5.6.1 Dependency Injection integrata

La registrazione avviene prima di costruire l'applicazione:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddSingleton<IClock, SystemClock>();
builder.Services.AddHttpClient<PaymentClient>();
```

```
var app = builder.Build();
```

Il container integrato risolve i servizi e li libera secondo il lifecycle. I tre lifetime principali sono:

- **Transient:** nuova istanza a ogni risoluzione. Adatto a oggetti leggeri e senza stato condiviso.
- **Scoped:** una istanza per scope; nelle API web di norma uno scope per richiesta. `DbContext` è tipicamente scoped.
- **Singleton:** una istanza per l'intera durata del processo. Deve essere thread-safe e non conservare dati specifici della richiesta.

Il lifetime non definisce soltanto l'efficienza. Un singleton che dipende da uno scoped può catturare un oggetto oltre la sua durata (*captive dependency*). Un servizio scoped risolto dal singleton può accedere a un `DbContext` condiviso in modo non sicuro; due richieste concorrenti potrebbero interferire. L'abilitazione delle verifiche di scope in sviluppo aiuta a rilevare configurazioni errate, ma i test devono coprire il comportamento reale.

Iniettare attraverso il costruttore rende visibili le dipendenze:

```
public sealed class OrderService(
    IOrderRepository repository,
    IClock clock) : IOrderService
{
    public Task<Order?> FindAsync(
        int id, CancellationToken ct) =>
        repository.FindAsync(id, ct);
}
```

Una classe con moltissimi parametri può indicare che coordina troppe responsabilità. Non risolvere il problema iniettando il provider generico `IServiceProvider` e facendo service locator: le dipendenze diventano invisibili, l'errore emerge a runtime e i test richiedono più setup.

5.6.2 Configuration providers e priorità

La configurazione standard combina valori da più fonti, tipicamente file JSON, variabili d'ambiente, user secrets in sviluppo e argomenti CLI. I provider successivi possono sovrascrivere valori precedenti; conoscere l'ordine è essenziale per capire quale valore è effettivamente attivo.

```
{
  "Catalog": {
    "PageSize": 50,
    "EnableBetaSearch": false
  }
}
```

Le variabili d'ambiente usano convenzioni come `Catalog__PageSize=100` per rappresentare la gerarchia. `appsettings.json` contiene configurazione non segreta e valori predefiniti; non vi si inseriscono password, token o chiavi di produzione. La configurazione non è una stringa da leggere liberamente in ogni metodo: raggruppare opzioni correlate in tipi tipizzati, validarne i valori all'avvio e fallire esplicitamente se manca un requisito essenziale.

```
builder.Services
    .AddOptions<CatalogOptions>()
    .BindConfiguration("Catalog")
    .Validate(o => o.PageSize is > 0 and <= 200,
        "PageSize deve essere compreso tra 1 e 200.")
    .ValidateOnStart();
```

Un default silenzioso per una configurazione obbligatoria può produrre un'applicazione che si avvia ma invia dati all'host sbagliato o disabilita una protezione. Preferire un errore chiaro in fase di avvio e loggare il nome della chiave mancante, non il suo valore segreto.

5.6.3 Secrets e ambienti

In sviluppo, User Secrets evita di commettere segreti nel repository, ma non è un vault di produzione. In produzione, usare il secret manager della piattaforma o un provider protetto, con rotazione, accesso minimo e audit. Evitare segreti in immagini container, argomenti di processo e log.

Separare configurazione per ambiente senza duplicare interi file con valori divergenti. Il codice dovrebbe leggere la configurazione da un'unica interfaccia; è l'ambiente a fornire il valore. La pipeline deve poter distribuire lo stesso artefatto in ambienti diversi e applicare i parametri esternamente.

Una chiave di produzione non deve essere copiata su una macchina locale per "far funzionare" il test. Usare credenziali effimere, identità gestite o servizi sostitutivi con permessi limitati.

5.6.4 Stato: richiesta, sessione e cache

Lo stato di una richiesta HTTP vive nell'oggetto `HttpContext` e non va conservato oltre la richiesta. Il server può servire richieste concorrenti su più thread: campi statici o singleton mutabili non sono un sostituto della sessione.

Distinguere:

- **stato di dominio persistente:** database o sistema autorevole;
- **cache:** dato ricostruibile con scadenza, invalidazione e comportamento in caso di miss;
- **sessione:** dati temporanei legati a un utente, con implicazioni di privacy e scalabilità;

- **stato locale al processo:** utile per ottimizzazioni effimere, mai fonte di verità in un cluster.

Con più istanze, cache o sessioni in memoria divergono. Una richiesta successiva può arrivare a un nodo diverso; sticky sessions riducono alcuni sintomi ma non sostituiscono storage distribuito e resilienza. Documentare durata, chiave, invalidazione, consistenza e comportamento in caso di indisponibilità della cache.

Non mettere nella cache dati sensibili o per-utente usando chiavi non isolate. Stabilire un limite di memoria e TTL; un cache stampede può saturare database e servizi proprio quando una chiave scade. Le strategie di refresh in background, lock distribuiti o stale-while-revalidate vanno scelte in base alla tolleranza alla staleness.

5.6.5 Client HTTP e integrazione

Per chiamare servizi esterni usare `IHttpClientFactory` o client tipizzati; evitare di creare e distruggere `HttpClient` a ogni chiamata. Configurare base address, timeout, handler e credenziali in modo centralizzato. Un client tipizzato rende visibili gli endpoint esterni e facilita test, logging e policy di resilienza.

Non registrare token nei log. Configurare timeout sia per la richiesta sia per operazioni specifiche; un timeout infinito può accumulare richieste e socket. Distinguere errori DNS, connessione rifiutata, timeout, status non riuscito e contenuto non valido. La risposta di un sistema remoto è input non attendibile anche quando usa TLS.

5.6.6 Background services e lavoro durevole

`BackgroundService` avvia un task con il lifecycle dell'host. È adatto a polling o lavori periodici semplici, non è una coda persistente. Se serve consegna affidabile, retry dopo crash, scheduling distribuito o visibilità operativa, usare un broker o un job scheduler con storage durevole.

Quando il worker usa servizi scoped, creare uno scope per l'unità di lavoro:

```
public sealed class ImportWorker(
    IServiceScopeFactory scopeFactory,
    ILogger<ImportWorker> logger) : BackgroundService
{
    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await using var scope = scopeFactory.CreateAsyncScope();
            var importer = scope.ServiceProvider
                .GetRequiredService<IImporter>();
```

```
            await importer.ProcessBatchAsync(stoppingToken);
            await Task.Delay(TimeSpan.FromSeconds(10), stoppingToken);
        }
    }
}
```

In codice reale gestire cancellazione e backoff: un'eccezione non recuperata può terminare il worker o il processo secondo configurazione. Il batch deve essere ripetibile, limitato e osservabile. Non avviare un task fire-and-forget da una route, perché l'host non ne governa durata o errori.

5.6.7 Progettare le dipendenze come confini

Ogni dipendenza esterna dovrebbe essere descritta da un'interfaccia o da un adapter quando questo aiuta a isolare un confine reale: provider di pagamento, clock, file storage o API remota. Non serve creare un'interfaccia per ogni classe. L'obiettivo è sostituire un'integrazione durante test o migrazione senza nascondere le capacità utilizzate.

Annotare chi possiede la configurazione e il lifecycle. Nei test, usare servizi in-memory soltanto se preservano la semantica necessaria; un dizionario non simula transazioni o concorrenza di un database relazionale. La sostituzione di una dipendenza deve essere esplicita e non attivata per errore in produzione.

5.7 Dati e integrazione con i sistemi esistenti

Il database è spesso il vincolo più forte di una migrazione. L'applicazione nuova può essere distribuita in poche ore; i dati possono essere condivisi con altri servizi, avere anni di storia e contenere anomalie che nessun modello documenta. Pianificare insieme schema, codice, ownership, sincronizzazione e rollback.

5.7.1 EF Core e scelta dell'accesso ai dati

Entity Framework Core è un ORM che traduce query LINQ in operazioni sul database e tiene traccia delle entità caricate. È adatto a molti casi applicativi, ma non è obbligatorio per ogni query. SQL esplicito o un micro-ORM possono essere più chiari per report, query complesse o requisiti di prestazione. La scelta va fatta per workload e misurazioni, non per fedeltà all'ORM sorgente.

Concetto di altri ORM	EF Core	Verifica necessaria
Session/entity manager	`DbContext`	Scope, tracking e concorrenza
Entity/model	Entità CLR	Identità, navigazioni e mapping

Concetto di altri ORM	EF Core	Verifica necessaria
Schema migration	EF migrations	SQL generato e deploy sicuro
Lazy loading	Opzionale, non predefinito in ogni scenario	Query implicite e N+1
Unit of work	`SaveChanges` / transazione	Confini e atomicità

`DbContext` è un'unità di lavoro breve, normalmente scoped per richiesta. Non è thread-safe: non lanciare query concorrenti sullo stesso contesto con `Task.WhenAll`. Usare query proiettate per leggere solo i campi necessari e `AsNoTracking` quando il risultato non verrà aggiornato tramite il context.

```
var products = await db.Products
    .AsNoTracking()
    .Where(p => p.IsActive)
    .OrderBy(p => p.Name)
    .Select(p => new ProductListItem(
        p.Id, p.Name, p.Price, p.Currency))
    .ToListAsync(cancellationToken);
```

La proiezione esplicita rende visibile il contratto della query e riduce trasferimento di dati. Non enumerare query infinite o materializzare grandi tabelle senza paginazione. `IQueryable` esegue lavoro quando enumerato; la query va mantenuta nel livello che conosce il database per evitare sorprese.

5.7.2 Modello, schema e confini

Non assumere che la struttura del modello sorgente si trasferisca direttamente. Identificare chiavi primarie e naturali, vincoli, nullability, precisione dei numeri, timezone, collations, indici, relazioni e politiche di cancellazione. Confrontare il tipo logico e il tipo fisico: `decimal` può avere scale diverse, un timestamp può essere locale oppure UTC, un `string` può avere limiti e collation.

Separare modello persistito e DTO quando la forma pubblica o le regole del dominio differiscono dalla tabella. Non usare il database come unico documento del dominio, ma non duplicare ogni campo in modelli senza un confine utile. Le regole di integrità devono essere applicate anche nel database per proteggere da race condition e percorsi non HTTP.

Prima di introdurre un nuovo database, capire quali altri componenti leggono o scrivono lo schema attuale. Uno schema condiviso è un contratto implicito. Migrare un servizio lasciando integrazioni non documentate può interrompere sistemi che non fanno parte del repository.

5.7.3 Migrazioni dello schema

Le migrazioni EF Core descrivono evoluzioni dello schema; non sostituiscono revisione e piano di deploy. Ispezionare lo SQL generato, capire lock e durata, e distinguere modifiche additive da modifiche distruttive. Un `ALTER TABLE` può bloccare una tabella grande; il comportamento dipende dal provider e dalla versione del database.

Un pattern compatibile è **expand–migrate–contract**:

1. aggiungere colonna o tabella compatibile con il codice precedente;
2. distribuire codice che legge/scrive nel formato compatibile;
3. copiare o trasformare i dati a batch e verificare conteggi;
4. passare la lettura al nuovo campo e osservare;
5. rimuovere il vecchio campo solo quando nessun consumer o rollback lo richiede.

Non eseguire automaticamente migrazioni potenzialmente lunghe all'avvio di ogni replica. Le istanze possono partire insieme e competere per lock; il processo applicativo potrebbe non avere permessi DDL in produzione. Preferire un passaggio controllato in pipeline o un job di deployment con un solo proprietario e log chiari.

Per un rollback, la migrazione inversa deve essere realmente possibile. Se il nuovo codice scrive valori non rappresentabili nel vecchio schema, il downgrade non è sicuro. Conservare backup testati e definire una strategia di ripristino coerente con RPO e RTO; un backup mai ripristinato non è evidenza sufficiente.

5.7.4 Migrare i dati

Il piano dipende da volume, downtime consentito e possibilità di mantenere sincronizzati i sistemi. Opzioni comuni:

- **Finestra di fermo:** bloccare scritture, esportare, trasformare, importare, verificare e riaprire. Semplice, ma richiede una finestra accettabile.
- **Snapshot e delta:** copiare una fotografia iniziale, poi applicare le modifiche intervenute fino al cutover. Richiede un meccanismo affidabile per identificare il delta.
- **Replica/CDC:** acquisire cambiamenti dal log del database o da un processo dedicato. Riduce il fermo, ma aggiunge dipendenze e necessità di controllo della consistenza.
- **Migrazione per partizioni/tenant:** trasferire porzioni indipendenti e cambiare ownership progressivamente. Funziona solo se il confine è stabile.

Per ciascuna opzione descrivere il punto di consistenza, il comportamento delle scritture durante la copia, l'ordine delle trasformazioni e come riprendere dopo un'interruzione. Gli script devono essere idempotenti o avere checkpoint

persistenti. Registrare errori per record e consentire la ripresa senza duplicare dati.

5.7.5 Qualità e semantica dei dati

I dati storici possono violare le regole attuali: stringhe vuote al posto di null, date impossibili, duplicati, codici fuori dominio, valute miste o relazioni orfane. Non correggere silenziosamente senza una regola approvata. Classificare gli elementi in validi, trasformabili, da mettere in quarantena e bloccanti.

Per date e orari distinguere istante UTC, orario locale e data civile. `DateTime` senza `Kind` esplicito può essere ambiguo; `DateTimeOffset` preserva offset ma non identifica da solo il fuso orario nominato. Convertire secondo il significato di business, non aggiungendo semplicemente `Z` alle stringhe.

Per denaro definire valuta, scala e arrotondamento; non usare floating point binario per valori monetari senza una motivazione. Per testo considerare Unicode, normalizzazione e collation: due stringhe visivamente identiche possono avere rappresentazioni differenti. Per identificativi preservare zeri iniziali se fanno parte del valore.

5.7.6 Riconciliazione verificabile

Il confronto tra origine e target deve andare oltre il conteggio righe. Verificare:

- conteggi per tabella, tenant e intervallo temporale;
- checksum o hash di colonne canoniche;
- riferimenti e vincoli;
- aggregati di business, per esempio totali per stato;
- campioni mirati di record complessi;
- dati modificati durante il delta e record scartati.

Definire tolleranze solo dove la trasformazione le giustifica, come precisione numerica o normalizzazione. Conservare report firmati o immutabili del processo, con timestamp, versione dello script e conteggi. Un esito “completato” senza numeri è difficile da auditare.

5.7.7 Transazioni, concorrenza e query

Una transazione database protegge un insieme di modifiche all’interno del provider e del suo contesto; non rende atomica una chiamata HTTP più una scrittura. Non tenere una transazione aperta mentre si attende un servizio remoto. Per coordinare DB ed eventi usare pattern come outbox con consumer idempotente.

Gestire concorrenza ottimistica con token o timestamp e restituire un conflitto comprensibile quando una versione è cambiata. Testare query reali con il provider di produzione: SQLite in memoria può differire per tipi, collations, transazioni e funzioni. I test rapidi con provider sostitutivo sono utili ma non sostituiscono test d’integrazione sul motore effettivo.

Misurare query e piani d'esecuzione prima di aggiungere repository o cache. Attenzione a N+1, caricamento eccessivo, assenza di indici e paginazione con offset su dataset enormi. Non interpolare SQL con input; usare parametri e permessi minimi.

5.7.8 Connessioni e continuità

Le stringhe di connessione provengono da configurazione protetta. Impostare pool, timeout, TLS e retry secondo provider; una connessione esaurita può apparire come lentezza dell'API. Il pool riduce l'overhead ma non aumenta la capacità del database: moltiplicare repliche moltiplica anche le connessioni potenziali.

Durante la convivenza, definire quale sistema può scrivere ciascuna entità e come viene mantenuta la consistenza. Evitare di far dipendere la nuova API direttamente da uno schema che il vecchio sistema può cambiare senza coordinamento. Un contratto dati versionato o un adapter temporaneo può isolare la migrazione e rendere esplicita la dipendenza da rimuovere.

5.8 Sicurezza e identità

La sicurezza non è un passaggio finale di hardening. Cambiare framework e identità modifica cookie, token, chiavi, policy e trust boundary. Una migrazione può mantenere gli endpoint e allo stesso tempo disconnettere gli utenti, accettare token non validi o rivelare dati. Definire i requisiti e verificare il flusso end-to-end prima del cutover.

5.8.1 Iniziare da threat model e inventario

Identificare asset (dati personali, credenziali, pagamenti, sessioni), attori, confini di fiducia e minacce rilevanti. Per ogni flusso chiedersi chi può invocarlo, come viene autenticato, quali permessi servono, quali input controlla e cosa accade se la dipendenza fallisce.

Mappare:

- provider di identità e modalità di login;
- cookie, sessioni, token access e refresh, durata e revoca;
- ruoli, claims, tenant e regole d'autorizzazione;
- segreti applicativi, chiavi di firma e certificati;
- dati sensibili nei log, backup, cache e telemetria;
- protezioni esistenti, dipendenze vulnerabili e procedure d'incidente.

Le policy scritte nel codice possono essere incomplete rispetto alla configurazione del gateway o del provider. Verificare la catena completa, inclusi reverse proxy e client. Fare approvare i cambiamenti da chi è responsabile di sicurezza e identità.

5.8.2 Autenticazione e autorizzazione non sono la stessa cosa

L'autenticazione stabilisce l'identità; l'autorizzazione decide cosa quell'identità può fare. In ASP.NET Core la pipeline usa handler di autenticazione e policy:

```
builder.Services.AddAuthentication()
    .AddJwtBearer();

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CatalogEditors", policy =>
        policy.RequireClaim("permission", "catalog.write"));
});

app.UseAuthentication();
app.UseAuthorization();

app.MapPost("/api/products", CreateProduct)
   .RequireAuthorization("CatalogEditors");
```

Il frammento mostra la forma generale, non una configurazione JWT completa: emittente (`iss`), destinatario (`aud`), chiavi, HTTPS, scarto orario e validazione dei claim vanno configurati secondo l'identity provider. Non disabilitare la validazione della firma o del destinatario per far passare un test.

Ruoli e claims hanno semantiche diverse. Un ruolo descrive appartenenza a una categoria; un claim può rappresentare un attributo o permesso. Non affidarsi a un claim ricevuto da un client non autenticato. Per multi-tenant, verificare che l'identificativo tenant sia vincolato all'identità e che ogni query applichi il perimetro corretto.

5.8.3 Migrare password e account

Gli hash delle password non sono password reversibili. Se il sistema nuovo usa un algoritmo o formato diverso, le opzioni includono:

- mantenere temporaneamente la verifica tramite il sistema identità esistente;
- validare al primo login con il vecchio verificatore e aggiornare l'hash in modo sicuro;
- richiedere reset password se la verifica compatibile non è disponibile;
- migrare verso un identity provider condiviso con processo supportato.

Non abbassare il costo dell'hash o introdurre una password universale per semplificare il passaggio. Non esportare hash in file non protetti. Proteggere il flusso di reset, limitare tentativi e non rivelare se un account esiste. Pianificare account bloccati, MFA, utenti disabilitati e audit.

5.8.4 Cookie, token e continuità di sessione

I cookie devono avere `Secure`, `HttpOnly` e una policy `SameSite` coerente con i flussi di login. Se contengono credenziali, gestire CSRF per richieste che modificano stato. La protezione CSRF non è sostituita da CORS.

JWT è un formato di token, non una strategia di autorizzazione completa. Validare firma, emittente (`iss`), destinatario (`aud`), scadenza e algoritmo consentito; limitare durata e proteggere refresh token. Non usare token come sessione revocabile senza un meccanismo di revoca o rotazione. Ridurre i dati sensibili contenuti nel payload: il payload firmato non è necessariamente cifrato.

I cookie di ASP.NET Core Identity o la Data Protection API possono essere specifici dell'applicazione e delle chiavi usate. Se durante il rollout vecchio e nuovo sistema devono accettare lo stesso cookie, verificare esplicitamente formato, scopo, chiavi e compatibilità supportata. Non improvvisare la condivisione delle chiavi: comprometterle può consentire la forgiatura di cookie o la decifratura di dati protetti.

5.8.5 Data Protection API e chiavi

ASP.NET Core Data Protection protegge payload applicativi, inclusi cookie e token antiforgery. In sviluppo le chiavi possono essere locali; in produzione le istanze devono avere una configurazione durevole e coerente quando devono decifrare gli stessi payload. Conservare le chiavi in storage protetto, limitarne l'accesso, pianificare rotazione e backup e impostare un application name appropriato per isolare applicazioni.

Un container effimero non deve essere l'unico luogo dove risiedono chiavi che servono dopo il riavvio. D'altra parte, condividere indiscriminatamente lo stesso key ring fra applicazioni aumenta l'impatto di una compromissione. Testare restart, scale-out e rotazione in un ambiente simile alla produzione.

5.8.6 Segreti e configurazione sicura

Usare identità gestite o secret store quando disponibili, permessi minimi, rotazione e audit. Evitare chiavi in `appsettings.Production.json`, Dockerfile, immagini, repository o log CI. Un secret presente nella history Git va considerato compromesso: rimuoverlo dal file non revoca la credenziale.

Separare segreti e configurazione ordinaria, ma non trattare un URL interno o un nome utente come sicuro per definizione. Tutto ciò che compare nelle eccezioni o negli header può finire in un log. Definire redazione e classificazione dei dati per log, tracing e metriche.

5.8.7 Difese per API web

- Validare input sul server, limitare dimensione e complessità delle richieste.

- Usare query parametrizzate; non costruire SQL concatenando input.
- Codificare output secondo il contesto e non fidarsi di dati già salvati.
- Configurare CORS con origini esplicite; non è un controllo di accesso per client non-browser.
- Impostare TLS e forwarded headers solo per proxy fidati.
- Applicare rate limiting, timeout e limiti di concorrenza appropriati.
- Ridurre permessi di database, file system e identità cloud.
- Mantenere pacchetti e immagini aggiornati e scansionare le dipendenze.

I middleware di sicurezza devono essere ordinati e testati in hosting reale. Una policy CORS corretta in locale può essere diversa dietro gateway. Non esporre stack trace in produzione; associare l'errore a un correlation id che il supporto può cercare.

5.8.8 Test di sicurezza nella transizione

Provare almeno: anonimo su route protetta; utente autenticato senza permesso; permesso valido; token scaduto, malfirmato o con destinatario (`aud`) errato; tenant diverso; cookie senza protezione CSRF; credenziale revocata; limite di rate; input e payload eccessivi. Verificare che gli errori non rivelino dati e che i log non contengano token o password.

Mantenere una matrice delle policy per endpoint. Ogni route deve avere una decisione esplicita: anonima, autenticata o protetta da policy nominata. Le route nuove non devono diventare anonime per omissione. Integrare test automatici e una verifica di configurazione in CI.

5.8.9 Cutover dell'identità

Durante una migrazione incrementale è possibile che alcuni endpoint usino il vecchio provider e altri il nuovo. Definire chi emette il token, chi lo valida e quali claims sono autorevoli. Se si usa un gateway per tradurre identità, documentare i trust boundary e impedire che il client possa falsificare gli header interni.

Preparare una strategia per sessioni non trasferibili: riautenticazione comunicata agli utenti, durata della transizione, revoca delle sessioni e supporto. Misurare i login falliti dopo il cutover e predisporre un rollback che non riattivi credenziali compromesse. La continuità non giustifica mantenere per sempre un algoritmo o un cookie legacy insicuro.

5.9 Testing e qualità

La migrazione è l'occasione per rendere esplicito il comportamento del sistema, non per sostituire i test del vecchio stack con test che verificano solo dettagli interni del nuovo. I test più preziosi descrivono ciò che client e utenti possono osservare, e permettono di confrontare le due implementazioni sugli stessi casi.

5.9.1 Test di caratterizzazione

Un test di caratterizzazione registra il comportamento attuale, anche quando la documentazione è incompleta. Si preparano input controllati, si invocano route o funzioni del sistema sorgente e si salvano status, header e payload normalizzati. Questi test diventano una specifica di compatibilità per il target.

La normalizzazione va limitata a campi realmente variabili, come timestamp generati o identificativi casuali. Normalizzare interi payload può nascondere regressioni. Per dati sensibili usare fixture sintetiche o anonimizzate; non importare dump di produzione nei test senza autorizzazione e protezioni.

Per endpoint rappresentativi raccogliere:

- risposta valida e codici di successo;
- input mancante, malformato e ai limiti;
- risorsa assente e conflitto;
- chiamante anonimo e permessi insufficienti;
- timeout o indisponibilità di una dipendenza;
- richieste duplicate e concorrenza.

Il test di caratterizzazione non dichiara che il comportamento corrente sia ideale. Un bug noto può essere registrato come comportamento da cambiare, con un test che viene aggiornato nella stessa modifica della decisione e con l'approvazione del proprietario del contratto.

5.9.2 Piramide e livelli di test

Unit test verificano funzioni e regole isolate, rapidamente e senza rete. Sono adatti a validazioni, calcoli, policy e transizioni di stato. Non devono ricreare l'intero framework con mock che duplicano l'implementazione.

Test d'integrazione esercitano più componenti: hosting, route, serializzazione, DI, database e adapter. In ASP.NET Core `WebApplicationFactory<TEntryPoint>` avvia l'applicazione nel processo di test e permette richieste HTTP in memoria. Sostituire configurazione o dipendenze in modo controllato; verificare anche la pipeline vera.

Test end-to-end attraversano deployment, gateway e servizi integrati. Sono più lenti e sensibili all'ambiente; riservarli ai flussi business critici, mantenere dati isolati e avere pulizia affidabile.

La piramide è una guida alla velocità e al feedback, non una quota rigida. Un test d'integrazione diretto sul database può essere più attendibile di un grande insieme di mock che non simula transazioni e SQL.

5.9.3 Test HTTP con WebApplicationFactory

Un test concettuale può verificare un contratto HTTP:

```
public sealed class ProductApiTests(
    WebApplicationFactory<Program> factory)
    : IClassFixture<WebApplicationFactory<Program>>
{
    [Fact]
    public async Task Missing_product_returns_not_found()
    {
        using var client = factory.CreateClient();
        using var response =
            await client.GetAsync("/api/products/987654");

        Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
    }
}
```

Il progetto deve esporre il punto d'ingresso al test se necessario e configurare un database isolato. Non riutilizzare un database condiviso tra esecuzioni concorrenti senza namespace o transazioni pulite. Per test di persistenza, verificare vincoli, migrazioni e comportamento del provider adottato in produzione.

Testare le risposte come farebbe un client: status, content type, header, schema e valori. Non verificare il nome di una classe interna quando l'obiettivo è la compatibilità API. Test specifici di unità possono invece verificare il dominio senza attraversare HTTP.

5.9.4 Parità differenziale

Quando entrambi i sistemi sono avviabili, inviare lo stesso caso a sorgente e target e confrontare gli output. Il test differenziale può essere automatizzato su una collezione di casi deterministici. Escludere o normalizzare solo campi non deterministici, mantenendo visibili differenze nei codici, nei valori e negli errori.

Non inviare automaticamente richieste di scrittura reali a entrambe le versioni: potrebbero creare effetti duplicati, inviare email o addebitare un pagamento. Per i casi mutativi usare database isolati, simulazioni senza effetti collaterali o una modalità di shadow che calcola la risposta senza committare.

Il confronto deve considerare anche proprietà mancanti e ordine delle liste quando l'ordine è contrattuale. Per importi, date e testo, confrontare con regole esplicite; non applicare una tolleranza generica che renda irrilevanti differenze di business.

5.9.5 Dipendenze nei test

Mock e fake hanno valore quando isolano un confine, per esempio un servizio di pagamento. Non mockare il database con una collezione in memoria e concludere che le transazioni reali funzionano. Usare container effimeri o istanze de-

dicate per testare provider SQL, code o identity server quando la compatibilità è importante.

Ogni fake deve avere semantica dichiarata. Un fake cache senza scadenza non verifica TTL; un fake storage senza errori non prova retry; un fake repository che restituisce sempre l'entità non esercita i vincoli. Preferire test a contratto per verificare che adapter diversi rispettino lo stesso protocollo.

5.9.6 Flakiness, dati e parallelismo

Un test che dipende dall'orologio reale, da un endpoint esterno o da dati condivisi può essere intermittente. Iniettare un clock quando il tempo è parte del comportamento, controllare il seed e isolare risorse. Non disabilitare l'esecuzione parallela dell'intera suite per nascondere una race: identificare lo stato condiviso e isolare il test interessato.

I test devono poter essere ripetuti. Ogni esecuzione dovrebbe creare e rimuovere i propri dati, usare nomi univoci e produrre log utili in caso di fallimento. Non riutilizzare credenziali di produzione. La cleanup deve funzionare anche dopo un errore, senza cancellare database o namespace condivisi.

5.9.7 Qualità e pipeline CI

Una pipeline minima esegue restore, build Release, test, analisi e pubblicazione di artefatti. Aggiungere scansione dipendenze, formattazione e analisi statiche secondo il rischio e gli standard del team. Bloccare la release su fallimenti significativi e rendere i risultati rintracciabili con commit, versione SDK e configurazione.

```
dotnet restore
dotnet format --verify-no-changes
dotnet build -c Release --no-restore
dotnet test -c Release --no-build --logger "trx"
dotnet publish src/Catalogo.Api -c Release -o artifacts/publish
```

La pipeline deve funzionare da checkout pulito e non dipendere da file generati presenti sulla workstation. Separare test che richiedono servizi esterni e rendere espliciti prerequisiti e cleanup. Verificare che il pacchetto pubblicato sia lo stesso che verrà promosso, non ricostruire da sorgenti diversi in ogni ambiente.

5.9.8 Copertura orientata al rischio

La percentuale di line coverage non misura quanto i test proteggono il contratto. Dare priorità a flussi con impatto alto, transizioni di stato, permessi, calcoli finanziari, migrazione dati e fallback. Un singolo test di integrazione ben progettato può coprire un rischio non espresso da decine di test di getter.

Per ogni capacità migrata, associare requisito, test, output e owner. I criteri di uscita possono richiedere: test di parità superati; regressioni spiegate; contratti

aggiornati; test sul provider reale; nessun difetto bloccante; report di qualità archiviato. Se una parte non è testabile, registrare il rischio e una mitigazione alternativa.

5.10 Prestazioni e comportamento del runtime

Una migrazione cambia runtime, librerie, scheduling, serializer e modalità di accesso ai dati. Il nuovo servizio può risultare più efficiente, equivalente o peggiore: la risposta si misura sul workload reale. Ottimizzare sulla reputazione del linguaggio o su un benchmark isolato porta a decisioni costose e poco attendibili.

5.10.1 Baseline e carichi confrontabili

Scegliere scenari che rappresentino uso, picchi e operazioni costose: lettura cache hit, miss, query DB, payload grande, dipendenza lenta e endpoint mutativo. Usare dataset e distribuzione dei dati confrontabili; una tabella con mille righe non simula una con milioni.

Misurare latenza p50/p95/p99, throughput, errori, CPU, memoria, allocazioni, connessioni e tempi di avvio. Includere warm-up e durata sufficiente a osservare garbage collection e saturazione delle dipendenze. Separare misure del client e del server: una rete più lenta non è una regressione del codice applicativo.

Per ogni test annotare versione runtime, build Release, hardware, container limits, numero istanze, configurazione, seed e commit. I confronti devono essere ripetibili; se il rumore è alto, raccogliere più campioni e indicare intervalli, non il singolo valore migliore.

5.10.2 Asincronia e concorrenza

ASP.NET Core gestisce molte richieste concorrenti senza assegnare necessariamente un thread dedicato a ogni attesa I/O. `async/await` consente di restituire il thread mentre si aspetta rete o disco. Non rende un calcolo CPU-bound più rapido, né elimina i limiti di database e upstream.

Evitare `.Result`, `.Wait()` e `GetAwaiter().GetResult()` sul percorso HTTP: il blocco occupa thread e sotto carico può causare starvation. Propagare `CancellationToken` fino ai client e alla persistenza. Se il client annulla la richiesta, il server dovrebbe interrompere lavoro inutile quando è sicuro farlo.

Non avviare operazioni parallele sullo stesso `DbContext`. Se più query indipendenti sono davvero necessarie, usare scope/contesti distinti con limite di concorrenza e verificare che il database possa sostenere il carico. Il parallelismo indiscriminato peggiora la latenza quando satura pool e connessioni.

5.10.3 Memoria e garbage collection

Il runtime .NET usa garbage collector generazionale; allocazioni temporanee sono spesso economiche, ma allocazioni eccessive aumentano pressione GC e latenza. Profilare prima di introdurre pooling o strutture più complesse. Riutilizzare buffer in modo scorretto può causare corruzione o esposizione di dati tra richieste.

Evitare di leggere file o payload illimitati in memoria. Applicare limiti, streaming e backpressure dove appropriato. Cache, singleton e collezioni statiche possono trattenere oggetti per tutta la vita del processo; monitorare dimensioni e politiche di eviction. Un processo containerizzato deve rispettare limiti memoria: essere stabile sulla workstation non prova che lo sia con un memory limit ridotto.

5.10.4 Query, serializzazione e caching

La causa dominante di lentezza è spesso I/O o query non efficienti. Usare log delle query e strumenti del database per diagnosticare N+1, full scan, lock e attese. Proiettare i campi necessari, paginare, aggiungere indici motivati dal piano e misurare dopo la modifica.

La serializzazione JSON può diventare significativa per payload grandi o grafi complessi. Definire DTO, evitare cicli nelle navigazioni e misurare allocazioni e dimensione della risposta. Cambiare serializer o opzioni di naming può rompere il contratto, anche se migliora le prestazioni.

La cache è utile quando riduce lavoro ripetuto e la staleness è accettabile. Definire TTL, invalidazione, chiave, isolamento tenant, limite memoria e comportamento in caso di errore. Misurare hit ratio e costo di riempimento. Non aggiungere cache per compensare query senza aver compreso la causa.

5.10.5 Profiling e Native AOT

Usare profiler e tracing per individuare il tratto costoso, confrontando CPU sample, allocation, thread pool e dipendenze. Un flame graph aiuta a trovare hot path, ma va letto insieme a metriche applicative. Gli strumenti di tracing devono essere configurati per minimizzare overhead e non registrare payload sensibili.

Trimming e Native AOT possono ridurre footprint o tempo di avvio per scenari specifici, ma impongono vincoli a reflection, serializzazione, plugin e librerie. Abilitare l'analisi e correggere warning, non silenziarli indiscriminatamente. Misurare cold start, dimensione e throughput nel target effettivo. Non assumere che AOT migliori un servizio dominato da I/O o database.

5.10.6 Prestazioni come criterio di cutover

Definire budget per operazione e soglie di rollback prima del rollout. Durante canary confrontare gruppi equivalenti per traffico, carico e dipendenze. Osservare code, errori e saturazione, non soltanto latenza media. Se il target peggiora, poterlo disattivare deve essere più rapido che applicare una patch d'emergenza.

La decisione finale non è "più veloce a ogni costo". Un sistema con latenza entro budget, meno incidenti e costo prevedibile può essere preferibile a un microbenchmark migliore ma a scapito di complessità e manutenibilità.

5.11 Container, CI/CD e produzione

La migrazione termina quando il sistema è esercibile dal team che lo mantiene: build riproducibile, artifact identificabile, configurazione esterna, deploy osservabile, rollback eseguibile e responsabilità di supporto chiare. Un endpoint che funziona sul computer dello sviluppatore non è ancora un servizio migrato.

5.11.1 Pubblicazione e artefatti

`dotnet publish -c Release` prepara output per un runtime e un ambiente di distribuzione. La pipeline deve compilare una volta e promuovere lo stesso artefatto tra gli ambienti, aggiungendo configurazione esterna. Ricostruire a ogni stage può produrre binari differenti senza che il team se ne accorga.

Registrare commit, versione SDK, dipendenze e provenance dell'immagine. Per applicazioni framework-dependent, il server deve avere un runtime compatibile; per self-contained, il pacchetto include runtime e richiede patching e dimensioni gestite diversamente. La scelta va documentata in relazione a hosting e policy di aggiornamento.

5.11.2 Dockerfile multi-stage

Un Dockerfile multi-stage separa compilazione e runtime:

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY ["src/Catalogo.Api/Catalogo.Api.csproj", "src/Catalogo.Api/"]
COPY ["src/Catalogo.Infrastructure/Catalogo.Infrastructure.csproj",
 ↪  "src/Catalogo.Infrastructure/"]
RUN dotnet restore "src/Catalogo.Api/Catalogo.Api.csproj"
COPY . .
RUN dotnet publish "src/Catalogo.Api/Catalogo.Api.csproj" \
    -c Release -o /app/publish --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish .
```

```
USER $APP_UID
ENTRYPOINT ["dotnet", "Catalogo.Api.dll"]
```

Adattare path, identità e immagini alla versione supportata e alla policy organizzativa. Non copiare segreti nel build context; usare `.dockerignore` per escludere `.git`, artefatti e file locali. Pinning e aggiornamento delle immagini devono essere governati: fissare un digest aumenta riproducibilità, ma richiede un processo per ricevere patch.

Eseguire con utente non root quando supportato; configurare porte e variabili dal runtime, non codificarle nel Dockerfile. Analizzare l'immagine, limitare capability e filesystem scrivibile, e verificare i certificati CA necessari alle chiamate HTTPS. Un container piccolo non è un container sicuro se contiene vulnerabilità non aggiornate.

5.11.3 CI/CD

La pipeline dovrebbe effettuare restore, build, test, analisi, pubblicazione, scansione, firma/provenance se prevista e deploy. I passaggi devono fallire in modo esplicito. Non stampare token in output e limitare i permessi del runner. Usare credenziali effimere o federate quando disponibili.

Separare build da deploy: il job di build produce artefatto immutabile; gli stage successivi applicano configurazione e promuovono lo stesso digest. Le approvazioni manuali possono essere necessarie per ambienti sensibili, ma non devono sostituire test automatizzati. Conservare log e report in un sistema accessibile al team durante incidenti.

La strategia di database va coordinata con la pipeline: le migrazioni non devono partire in parallelo da ogni replica. Le modifiche incompatibili richiedono deployment in più fasi. La pipeline dovrebbe rendere visibili script SQL generati e l'approvazione necessaria.

5.11.4 Reverse proxy e hosting

In produzione Kestrel può essere dietro IIS, Nginx, un ingress Kubernetes o un gateway cloud. Configurare forwarded headers da proxy fidati per ricostruire schema e IP originali. Se si accettano header forwarded da qualunque client, un chiamante può falsificare indirizzi o aggirare regole basate sull'IP.

Terminare TLS nel punto previsto, proteggere il collegamento tra proxy e applicazione quando richiesto e configurare redirect HTTPS coerenti. Verificare `PathBase` se il gateway pubblica l'app sotto un prefisso. Controllare upload size, timeout, buffering e idle timeout a ogni livello: il limite effettivo è spesso quello più basso della catena.

5.11.5 Health, osservabilità e shutdown

Distinguere:

- **liveness:** il processo è in grado di continuare a funzionare?
- **readiness:** l'istanza può ricevere traffico?
- **startup:** l'inizializzazione è completata?

Un health check che prova ogni dipendenza a ogni intervallo può amplificare un guasto. Separare controlli superficiali da diagnostica profonda e usare timeout. Non esporre endpoint di diagnostica dettagliati al pubblico.

Usare log strutturati con trace/correlation id, metriche aggregate e distributed tracing per seguire chiamate tra servizi. Evitare identificativi personali o token nei label delle metriche: cardinalità e privacy. Definire dashboard e alert collegati a obiettivi di servizio, con runbook e owner.

Lo shutdown deve cessare l'accettazione di nuovo traffico e consentire il completamento del lavoro entro il grace period dell'orchestratore. I worker devono ricevere cancellation token e fermarsi in modo controllato. Testare terminazione con richieste in corso e drain del load balancer.

5.11.6 Scalabilità, storage e resilienza

Un'applicazione web stateless è più facile da scalare, ma cache locale, sessione, chiavi Data Protection, file caricati e job in memoria introducono stato di istanza. Documentare dove vive ciascun dato e cosa succede se un nodo viene rimosso.

Usare storage condiviso per file che devono sopravvivere al container; non considerare il filesystem locale una fonte durevole. Se sono necessari sticky session, documentare i limiti e pianificare una migrazione verso stato distribuito quando opportuno. Limitare pool di connessioni in relazione al numero massimo di repliche e alla capacità database.

Resilienza non significa retry illimitato. Configurare timeout, retry limitati con backoff, circuit breaker o bulkhead secondo il failure mode. L'operazione deve avere semantica idempotente o un identificatore di deduplicazione. Un upstream degradato non deve esaurire thread, connessioni e code del servizio migrato.

5.11.7 Rollout progressivo e rollback operativo

Con canary o blue/green, spostare percentuali di traffico e confrontare errori e latenza su gruppi simili. Impostare soglie automatiche e un responsabile umano per l'arresto. Un rollout graduale senza metriche segmentate è solo un deploy lento.

Il rollback del codice deve essere compatibile con lo schema dati e con gli eventi già prodotti. Se i messaggi sono stati consumati o i dati trasformati, tornare

al binario precedente potrebbe non bastare. Preparare procedure per fermare consumer, drenare code, ripristinare dati o completare una compensazione.

Provare l'intero runbook in un ambiente rappresentativo: identificare la versione attiva, cambiare routing, confermare i segnali, annunciare lo stato e raccogliere evidenze. Registrare durata e impedimenti; assegnare le correzioni prima del cutover reale.

5.11.8 Costi, supporto e dismissione

Calcolare costo complessivo includendo doppio runtime durante la transizione, database, log, traffico, build runner, licenze e tempo operativo. La coesistenza dovrebbe avere una data o un criterio di uscita. Ogni dipendenza residua dal vecchio sistema va assegnata a un owner e classificata: necessaria, temporanea o da eliminare.

Prima di spegnere l'origine, verificare traffico residuo, job schedulati, webhook, client sconosciuti, backup, requisiti di audit e retention. Conservare solo ciò che è richiesto e con accessi limitati; spegnere credenziali e endpoint non più necessari. La riduzione della superficie d'attacco e del costo è parte del risultato della migrazione.

5.12 Caso pratico: migrare un servizio catalogo

Per collegare le decisioni, immaginiamo di migrare un servizio catalogo esistente, scritto in Python con FastAPI, verso ASP.NET Core. Il servizio pubblica prodotti, legge un database PostgreSQL condiviso con un back office e viene consumato da un sito e da un'app mobile. L'esempio non è un tutorial per ricostruire un catalogo completo: mostra come delimitare il lavoro, mantenere il contratto e produrre evidenze prima di spostare il traffico.

5.12.1 Scenario e vincoli

Il sistema attuale espone:

- `GET /api/products?category=...&page=...` per elencare prodotti attivi;
- `GET /api/products/{id}` per leggere un prodotto;
- `POST /api/products` per creare un prodotto, riservato agli editor;
- un job notturno che importa disponibilità da un partner.

Il database contiene milioni di righe storiche, mentre l'endpoint pubblico restituisce solo prodotti attivi. Il back office continua a scrivere nello schema durante la migrazione. L'app mobile non può essere aggiornata rapidamente. La disponibilità richiesta non consente un fermo lungo e l'autenticazione usa un identity provider aziendale.

L'assessment mostra che le letture pubbliche sono il primo candidato: sono frequenti, hanno contratto documentabile e possono rimanere read-only sul database esistente. La scrittura di prodotti resta sul servizio sorgente nella prima fase; non si duplicano gli effetti di `POST`. Il job notturno viene migrato separatamente perché ha un lifecycle e un profilo di retry diversi.

5.12.2 Baseline e test di caratterizzazione

Prima di scrivere il target si raccolgono:

- esempi di response per prodotto attivo, assente e non visibile;
- comportamento dei filtri e paginazione;
- status e schema per query non valide;
- requisiti di autorizzazione del `POST`;
- p95/p99, richieste al secondo, error rate e carico sul database;
- piano query corrente e indici usati.

Le fixture contengono dati sintetici con casi di null, descrizione lunga, prezzo al limite e categoria inesistente. Per i casi di produzione si usano payload anonimizzati, senza identificativi o informazioni personali. I test memorizzano anche ordinamento e proprietà JSON richieste dai client.

Si concorda che l'implementazione nuova deve restituire la stessa semantica dei filtri e i medesimi codici, salvo variazioni approvate. Il nuovo sistema deve mantenere il budget di latenza del servizio e non superare il limite di connessioni assegnato al database.

5.12.3 Creazione del progetto

La solution può iniziare con API e test, senza creare tutti i progetti architetturali possibili:

```
Catalogo.sln
src/
  Catalogo.Api/
  Catalogo.Infrastructure/
tests/
  Catalogo.Api.Tests/
```

Il progetto API configura hosting, autenticazione, logging, endpoint e composizione dei servizi. Infrastructure contiene `DbContext`, mapping e query SQL/EF Core. Se le regole del dominio diventano complesse, si può introdurre un progetto Domain con una decisione esplicita, non come prerequisito per un endpoint di lettura.

La foundation si considera pronta quando un commit pulito passa `dotnet restore`, `dotnet build`, `dotnet test`, produce l'immagine, gira nel namespace di test e legge configurazione esterna. Un health check minimale può essere registrato e mappato così:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddHealthChecks();

var app = builder.Build();
app.MapHealthChecks("/health");
app.Run();
```

Questo endpoint verifica che l'applicazione risponda; controlli di readiness sulle dipendenze vanno aggiunti solo se richiesti. La pipeline non usa una connection string reale nei log.

5.12.4 Contratto e query di lettura

Il DTO è separato dal modello persistito:

```
public sealed record ProductListItem(
    int Id,
    string Name,
    decimal Price,
    string Currency);

public sealed record ProductPage(
    IReadOnlyList<ProductListItem> Items,
    int Page,
    int PageSize,
    int TotalCount);
```

La paginazione deve essere limitata lato server; non accettare page size illimitata. Per dataset in crescita, una paginazione a cursore può essere più stabile ed efficiente di un offset elevato, ma cambiare il contratto va coordinato con i client. Nella prima fase si mantiene l'interfaccia esistente e si usa il piano database per verificare il costo.

Un service applicativo riceve filtri già validati e restituisce un risultato; l'endpoint mappa il risultato a HTTP:

```
app.MapGet("/api/products", async (
    string? category,
    int page,
    int pageSize,
    IProductQueries queries,
    CancellationToken ct) =>
{
    if (page < 1 || pageSize is < 1 or > 100)
        return Results.BadRequest();

    var result = await queries.ListAsync(
        category, page, pageSize, ct);
    return Results.Ok(result);
});
```

Il frammento illustra il flusso e non definisce da solo tutte le regole: i default per i parametri, il formato errore, i limiti e l'ordinamento vanno allineati al contratto sorgente. In codice di produzione la validazione restituisce una risposta coerente e documentata; l'endpoint non deve trasformare un errore database in lista vuota.

5.12.5 Mantenere il database condiviso in sicurezza

Il nuovo servizio apre un accesso read-only se il provider e i permessi lo consentono. Questo riduce il rischio di alterare dati durante la prima fase. Le query sono proiettate sui campi necessari, limitano risultati e usano parametri. Si controllano indici e piani; l'utente DB non ha permessi DDL o scrittura non necessari.

Prima del deploy si verificano schema e tipi con i proprietari del back office. Se il vecchio sistema cambia una colonna, il target deve fallire con log diagnostico e alert, non continuare con un risultato parziale apparentemente valido. Un contratto dati o una vista stabile può isolare il nuovo servizio dalle tabelle interne, se il team database può mantenerli.

Il modello EF Core mappa solo la parte necessaria dello schema, senza avviare migrazioni automatiche. Il target non è proprietario del database in questa fase; non può quindi presumere di aggiornare tabelle condivise. Le migrazioni future richiedono un piano separato con tutti i writer coinvolti.

5.12.6 Test differenziale e osservazione in shadow

La suite d'integrazione invia richieste al sistema sorgente e al target usando un database di test fissato. Le risposte sono confrontate su status, schema e valori. Timestamp e campi variabili sono normalizzati solo dove il contratto li dichiara non deterministici.

In ambiente di preproduzione il traffico sintetico esercita route e limiti. In produzione, se privacy, policy e architettura lo permettono, il gateway può duplicare **solo richieste di lettura** verso il target in modalità shadow. Il target esegue il calcolo ma la sua risposta non viene restituita all'utente. Si confrontano risultati in forma redatta e si misurano query e latenza.

Non duplicare il `POST` o il job di importazione: un secondo invio potrebbe creare un prodotto, inviare notifiche o alterare stock. Shadow traffic non è una licenza a replicare dati personali; minimizzare payload, applicare retention breve e ottenere le approvazioni necessarie.

Un report giornaliero classifica differenze per categoria: status, proprietà mancanti, ordinamento, valori numerici, null e timeout. Ogni differenza ha owner e decisione: bug nel nuovo codice, bug sorgente da correggere, differenza intenzio-

nale o dato invalido. Il traffico non viene spostato finché le differenze critiche non sono comprese.

5.12.7 Canary e cutover

Con test differenziali soddisfacenti, il gateway indirizza al target una piccola percentuale di richieste di lettura, mantenendo identificabile la coorte. Si controllano:

- error rate e status per route e versione;
- p95/p99 e timeout;
- pool connessioni, CPU, memoria e query lente;
- differenze di contenuto e errori di deserializzazione client;
- log di autenticazione e richieste senza tenant.

Si amplia gradualmente la percentuale solo se le metriche restano entro budget per una finestra definita. Il criterio non è "nessuna segnalazione": un client mobile può essere usato poco e segnalare tardi. Gli alert e i log devono offrire copertura tecnica indipendente dal feedback degli utenti.

Per rollback, il gateway riporta la route al servizio Python. Poiché il target è read-only e usa lo schema invariato, il rollback non richiede sincronizzare nuove scritture. Si prova prima in preproduzione, verificando cache, DNS, warm-up e propagazione della configurazione. Il piano include un referente che può arrestare il rollout.

5.12.8 Migrare la scrittura

Una volta stabile la lettura, la creazione prodotto diventa una capacità separata. Il team decide se il nuovo servizio assume la scrittura oppure se il vecchio sistema resta owner temporaneamente. Non basta duplicare le scritture: si deve evitare che back office e API sovrascrivano modifiche in modo inconsistente.

Se il target assume ownership, il rollout della scrittura avviene solo dopo aver verificato:

1. policy di autorizzazione e mapping claims;
2. validazione e invarianti condivise con il back office;
3. unicità e concorrenza nel database;
4. audit e notifiche prodotte una sola volta;
5. idempotenza delle richieste ritentate;
6. schema compatibile con la versione precedente durante il rollback.

Una chiave di idempotenza può aiutare per richieste ripetibili, ma deve avere storage e scadenza definiti. L'outbox può garantire che un evento venga registrato nella stessa transazione della creazione, con consegna asincrona e consumer deduplicante. Non rende transazionale il sistema esterno.

5.12.9 Migrare il job notturno

Il job è una capacità distinta dall'API. Si verificano frequenza, finestra, volume, checkpoint, dati parziali e comportamento al retry. Un `BackgroundService` può ospitare un ciclo semplice, ma se il job deve sopravvivere a riavvii, essere schedulato da più istanze o riprendere da un checkpoint, si usa un orchestratore o una coda durevole.

Durante la prova, il target importa in un ambiente isolato e confronta conteggi e aggregati con il job esistente. Non si eseguono entrambi in scrittura sullo stesso dataset senza una strategia d'idempotenza. Il cutover del job ha un lock o un singolo scheduler owner, un run id, report e procedura di replay.

5.12.10 Risultato e lezioni

Il servizio di lettura può essere considerato migrato quando il target ha sostenuto il traffico previsto, rispetta contratto e budget, è osservabile, ha un rollback provato e il sistema sorgente non riceve più quelle route. Il capitolo di progetto non si chiude però finché scrittura, job, integrazioni residue e dismissione non hanno owner e date.

Le decisioni hanno mantenuto il perimetro piccolo, ma non hanno ignorato il sistema completo. Si è iniziato con letture reversibili; dati condivisi e scritture sono rimasti sotto un owner; test e telemetria hanno preceduto il cutover; infine ogni capability ha avuto il suo piano. In un caso PHP o Java cambierebbero framework, tool e dettagli di persistenza, non il metodo di mappatura, caratterizzazione e rollout.

5.12.11 Adattare il caso ad altri ecosistemi

- **PHP/Laravel o Symfony:** inventariare middleware, service container, queue worker, session driver e convenzioni Eloquent/Doctrine. Verificare differenze di null, timezone e gestione dei job.
- **Python/Django o FastAPI:** distinguere ASGI/WSGI, dependency overrides, task asincroni, ORM e migration framework. Non assumere che codice `async` sia non bloccante se una libreria sottostante è sincrona.
- **Node.js/Express o NestJS:** mappare middleware e pipe/interceptor, controllare event loop, stream e dipendenze native dei pacchetti.
- **Java/Spring Boot:** confrontare lifecycle bean, scope request, filtri, transazioni e thread pool. Verificare semantica di ORM, lazy loading e gestione delle eccezioni.

Per ciascuno, costruire una scheda per concetto: comportamento sorgente, candidato ASP.NET Core, differenza osservabile, test di prova, impatto e decisione. Un mapping documentato riduce il rischio che una differenza venga scoperta solo da un utente in produzione.

5.13 Approfondire la mappa dai principali ecosistemi

I confronti tra ecosistemi diventano utili quando partono da un'applicazione concreta. Questa sezione raccoglie differenze ricorrenti che una migrazione dovrebbe provare esplicitamente. Non sono regole assolute: framework e versioni cambiano, e un progetto può adottare convenzioni diverse da quelle più comuni.

5.13.1 Da PHP e framework PHP

In una classica applicazione PHP, il modello di esecuzione può essere per richiesta: un web server inoltra la richiesta al runtime, che carica codice e bootstrap. Framework come Laravel o Symfony introducono container, middleware, routing, ORM, sessioni e console command. In .NET il processo web rimane tipicamente attivo e gestisce molte richieste durante la propria vita.

Questa differenza impone di cercare stato globale mutabile e risorse inizializzate una sola volta. Un singleton .NET non viene ricreato per ogni richiesta; non deve trattenere l'utente corrente o un `DbContext`. L'applicazione deve essere thread-safe e non affidarsi al teardown implicito che il processo PHP poteva offrire.

Mappare:

- route e controller PHP verso endpoint/controller ASP.NET Core, conservando metodi, status e formati;
- service container e service providers verso registrazioni DI con lifetime appropriato;
- middleware e kernel events verso pipeline ASP.NET Core, distinguendo codice globale da logica endpoint;
- Eloquent o Doctrine verso EF Core o SQL esplicito, verificando lazy loading, transazioni e naming;
- queue worker e scheduler verso broker, worker o orchestratore, non necessariamente `BackgroundService`;
- sessioni e file upload verso storage distribuito e limiti HTTP espliciti.

Un aspetto spesso trascurato è il tipo dinamico dei dati. PHP può convertire implicitamente stringhe numeriche, valori booleani o date; C# richiede parsing e conversioni più visibili. Non replicare coercizioni ambigue senza capire quali client ne dipendano. Per importare valori storici, definire cultura, decimal separator, encoding e regole per stringhe vuote.

5.13.2 Da Python e framework ASGI/WSGI

In Python distinguere il modello WSGI dal modello ASGI e capire se l'applicazione è davvero asincrona. Un endpoint `async` che chiama una libreria sincrona può bloccare l'event loop. In .NET, chiamare un metodo sincrono di I/O in

una route non viene reso asincrono dalla firma del delegate: occorre usare API asincrone della libreria o isolare il lavoro secondo un piano di capacità.

Django, Flask e FastAPI offrono convenzioni diverse per middleware, dependency override, serializzazione, migrazioni e background task. FastAPI genera OpenAPI da tipi e dichiarazioni, ma la specifica esistente va verificata contro le response effettive. Non assumere che il target generi gli stessi schema, descrizioni o default.

Per la persistenza, confrontare SQLAlchemy, Django ORM e l'eventuale SQL diretto con il modello EF Core. Verificare transazioni implicite, autoflush, session lifecycle, timezone e query lazy. Un Python `None` serializzato come `null` può diventare una proprietà assente se il serializer .NET è configurato diversamente.

Python consente spesso di distribuire sorgenti e dipendenze in un ambiente virtuale. In .NET l'artefatto publish, il runtime e il container sono parte della consegna. La pipeline dovrebbe ricreare l'output da checkout pulito, non riutilizzare un `venv` o un output locale. Le dipendenze native Python hanno analoghi problemi di piattaforma nelle librerie .NET con componenti nativi: provare l'immagine target, non soltanto il test locale.

5.13.3 Da JavaScript/Node.js

Node.js usa un event loop e un modello di I/O non bloccante. Il codice CPU-bound può bloccare l'event loop; in ASP.NET Core un lavoro CPU-bound può occupare thread del pool e saturare l'applicazione. In entrambi i casi servono limiti, profiling e un eventuale worker separato. Non trasferire un `setTimeout` o una promise non attesa come se fossero una coda affidabile.

Express compone middleware con convenzioni proprie, mentre NestJS introduce moduli, guard, pipe e interceptor. Mappare ciascuna responsabilità, senza trasformare ogni middleware in un servizio DI o ogni provider Nest in singleton .NET. Controllare l'ordine di middleware, la gestione degli errori asincroni e l'effetto delle route non corrispondenti.

TypeScript fornisce controlli statici durante la build ma i dati HTTP restano dinamici a runtime. C# offre un sistema statico integrato nel compilatore, tuttavia il binding JSON non sostituisce validazione. Non rimuovere i controlli runtime perché il DTO ha proprietà non-nullable: client non conformi possono inviare body incompleti.

Controllare differenze di serializzazione: `undefined` in JavaScript può essere omesso, `null` è distinto; in JSON non esiste `undefined`. Le opzioni di naming camelCase, enum come stringhe o numeri, date e proprietà ignorate devono essere definite e testate. Verificare inoltre gli stream: bufferizzare tutto il body può aumentare memoria e latenza rispetto a una pipeline che processava dati incrementali.

5.13.4 Da Java e Spring Boot

Java e .NET condividono alcuni concetti di runtime gestito, garbage collection, compilazione e applicazioni server long-running, ma il modello di librerie e framework resta specifico. In Spring, confrontare bean scope, filtri servlet, interceptor, controller advice, configurazione e transazioni con i componenti ASP.NET Core corrispondenti.

Un bean `singleton` Spring e un singleton DI .NET hanno entrambi una durata ampia, ma il comportamento effettivo dipende da proxy, scope e lifecycle del framework. Controllare le dipendenze scoped e la sicurezza concorrente. Non assumere che un `@Transactional` si traduca in una transazione che attraversa chiamate HTTP o eventi.

Confrontare JPA/Hibernate con EF Core su tracking, flush, lazy loading, cascade, lock ottimistico e query generation. Le due tecnologie possono produrre SQL diverso e differire nei limiti del provider. Provare query e migrazioni sullo stesso motore database; una conversione di annotation non è un piano dati.

Confrontare Maven/Gradle con NuGet e i file `.csproj` riguardo a versioni, dipendenze transitive, plugin e packaging. Verificare profili di build, variabili ambientali e configurazione del server. I test su Spring Test o container di test vanno sostituiti da test equivalenti sul target e sul provider effettivo, non semplicemente rimossi.

5.13.5 Tabella di mapping delle responsabilità

Durante l'assessment costruire una tabella concreta. Esempio:

Sistema sorgente	Comportamento osservato	Candidato .NET	Prova di migrazione
Middleware Laravel	Aggiunge tenant da un claim e blocca utenti sospesi	Middleware più policy o servizio endpoint	Test anonimo, utente sospeso e tenant valido
Dependency FastAPI	Fornisce repository per richiesta	Servizio scoped e repository/query service	Due richieste non condividono stato
Middleware Express	Trasforma errori asincroni in JSON	Exception handler centrale	Eccezioni note e inattese con status coerente
Filtro Spring	Verifica permesso di scrittura	Policy authorization	Token scaduto, claim mancante e claim valido
Job Celery	Import idempotente con retry	Worker e broker/job scheduler	Crash durante batch e replay senza duplicati

Sistema sorgente	Comportamento osservato	Candidato .NET	Prova di migrazione
Migrazione Doctrine	Aggiunge campo con backfill	Script controllato expand–migrate–contract	Rollout con vecchia versione ancora attiva

La colonna “candidato .NET” è un’ipotesi finché la prova non la conferma. Annotare anche alternative e decisione. Se il comportamento sorgente non è determinabile, creare prima un test o una sessione di discovery.

5.13.6 Scheda di decisione per un concetto

Una scheda riutilizzabile può avere questi campi:

```
Concetto sorgente:
Versione e configurazione:
Responsabilità osservata:
Input e output:
Lifecycle e scope:
Dipendenze e side effect:
Candidato ASP.NET Core:
Differenze semantiche:
Test che dimostra il mapping:
Decisione e motivazione:
Owner e data di revisione:
```

La scheda impedisce due errori opposti: tradurre tutto meccanicamente o ricominciare senza preservare nulla. I dettagli si possono archiviare vicino alla decisione tecnica e collegare ai test; il capitolo o la documentazione di progetto deve conservare almeno la motivazione e le incompatibilità ancora aperte.

5.13.7 Quando mantenere un componente nel sistema originario

Non tutto deve essere migrato immediatamente. Un motore di report con dipendenze non portabili, un adapter verso hardware o un job stabile possono restare temporaneamente dietro un contratto di servizio. La decisione va confrontata con il costo di mantenere runtime, competenze, security patch e pipeline separate.

Un componente mantenuto deve avere:

- owner, responsabile degli aggiornamenti e processo di incident response;
- interfaccia di rete o messaggio documentata, con autenticazione e timeout;
- monitoraggio e piano di continuità;
- dipendenze e dati che ne definiscono la vita residua;
- criterio e data di revisione della decisione.

La coesistenza è una strategia, non un risultato finale implicito. Se la migrazione completa non è conveniente, dichiarare il confine come architettura permanente e finanziarne la manutenzione; se è transitorio, misurarne la riduzione.

5.14 Prove di migrazione dei dati: runbook e controlli

Lo spostamento dei dati è un processo controllato con input, checkpoint e report. Il runbook seguente aiuta a trasformare uno script una tantum in un'attività ripetibile, verificabile e recuperabile.

5.14.1 Preparare il runbook

Prima di eseguire una copia reale, documentare:

1. database sorgente e target, versioni, region e responsabili;
2. oggetti e intervalli inclusi, esclusioni e regole di trasformazione;
3. permessi minimi delle identità usate e scadenza delle credenziali;
4. volume previsto, throughput, durata stimata e impatto sulle query;
5. punto di consistenza, strategia delta e trattamento delle scritture concorrenti;
6. checkpoint, retry, deduplicazione e ripresa dopo interruzione;
7. validazioni, soglie di errore, approvatore e piano di rollback;
8. retention e protezione dei file temporanei e dei report.

Il runbook non deve includere segreti in chiaro. I riferimenti alle credenziali devono indicare il secret store e l'identità autorizzata. Limitare accesso ai dump, cifrarli in transito e a riposo e cancellarli secondo retention approvata.

5.14.2 Snapshot, delta e freeze

Il piano di migrazione deve definire cosa significa "dati coerenti". Una fotografia estratta in più query a orari diversi può contenere righe incompatibili tra loro se il sistema continua a scrivere. Usare snapshot transazionali o un meccanismo CDC quando il provider lo supporta e la finestra di consistenza è necessaria.

Se si effettua un freeze:

- comunicare in anticipo il blocco e identificare tutte le write path, incluso back office;
- fermare job e consumer in ordine concordato;
- attendere che le transazioni in corso si completino;
- registrare watermark temporale o identificativo dell'ultimo record;
- estrarre, trasformare e importare;
- eseguire riconciliazione e test applicativi;
- riaprire le scritture una sola volta deciso il sistema owner.

Se il freeze non è possibile, usare uno snapshot più un delta ordinato e definire come trattare update e delete. Il timestamp `updated_at` da solo può non bastare se non è monotono, ha precisione insufficiente o non registra cancellazioni. Un log transazionale o una tabella outbox può fornire una sorgente di cambiamenti più affidabile.

5.14.3 Trasformazioni versionate

Ogni trasformazione deve essere identificata da una versione dello script e avere esempi prima/dopo. Separare parsing, validazione, mapping e scrittura per poter testare le regole senza connettersi ai database.

Esempi di trasformazioni che richiedono approvazione:

- stringa vuota convertita in `null`;
- codice paese normalizzato;
- date locali interpretate in un fuso nominato;
- enum legacy mappati a nuovi valori;
- importi con scala differente;
- record duplicati uniti secondo una regola di dominio.

Non inferire unità o timezone da un nome di campo ambiguo. Se un dato non può essere interpretato, metterlo in quarantena con ragione e chiave tecnica; non scartarlo silenziosamente. I report non devono esporre valori personali se bastano conteggio, categoria di errore e identificativo pseudonimo.

5.14.4 Importazione riprendibile

Per dataset grandi elaborare batch limitati, registrando il checkpoint in storage durevole. La chiave di ripresa deve rappresentare un ordine stabile, come una chiave primaria o un cursore monotono, non l'offset di una query che cambia mentre si importano righe.

Ogni batch può seguire questo ciclo:

1. leggere un intervallo consistente;
2. trasformare e validare in memoria limitata;
3. scrivere in transazione o staging;
4. verificare il batch;
5. registrare il checkpoint solo dopo il commit;
6. proseguire o fermarsi su soglia di errore.

Se il processo termina dopo il commit ma prima del checkpoint, il batch può essere ripetuto. Usare upsert con chiave stabile o una tabella di deduplicazione; non assumere consegna exactly-once. Il checkpoint prima del commit può invece saltare dati. Definire una strategia per ripetere in sicurezza.

5.14.5 Staging e cutover

Una tabella di staging consente di importare e validare senza esporre immediatamente i record agli utenti. Può contenere migration id, origine, timestamp e stato di validazione. Proteggere lo staging come il dato originale; non lasciarlo diventare un archivio permanente di copie non governate.

Il cutover richiede una decisione sulla lettura autorevole. Possibili percorsi: cambiare connection string, attivare un routing di tenant, promuovere tabelle o aggiornare un alias. Documentare la cache e la propagazione della configurazione, perché il cambio potrebbe non essere istantaneo.

Prima dell'attivazione, congelare le scritture secondo il piano oppure completare il delta, validare i criteri e verificare accesso e performance. Dopo l'attivazione, monitorare errori e mismatch per un periodo concordato; mantenere l'origine in sola lettura finché il rollback dati è ancora possibile.

5.14.6 Report di riconciliazione

Un report utile include:

Verifica	Origine	Target	Esito
Righe lette, scritte, scartate	conteggio	conteggio	riconciliato?
Record per stato e tenant	aggregato	aggregato	scostamento
Chiavi orfane	conteggio	conteggio	soglia
Hash di campioni canonici	hash	hash	corrispondenza
Valori non trasformabili	conteggio	quarantena	owner

I checksum devono essere calcolati su una rappresentazione canonica che definisce ordinamento, encoding e null. Un hash dell'intero file può rilevare differenze, ma non spiega quale record o trasformazione le abbia causate. Combinare aggregati, campioni e conteggi per localizzare rapidamente il problema.

Il report include migration id, commit dello script, configurazione non segreta, orari, watermark, durata e identità esecutrice. Conservare il report secondo policy e rendere riproducibili le query di verifica.

5.14.7 Criteri di stop

Interrompere l'importazione o il cutover se:

- gli errori superano la soglia concordata o coinvolgono dati critici;
- il delta non ha un watermark affidabile;
- la replica o il database supera il budget di carico;
- gli aggregati divergono senza una regola approvata;
- non è chiaro quale sistema possiede le scritture;
- il rollback richiesto non è più praticabile.

Fermarsi è un esito valido. Il runbook deve indicare come lasciare il sistema in uno stato consistente e chi decide se riprendere, ripristinare o applicare una correzione.

5.15 Diagnosi operativa durante la migrazione

Gli incidenti di migrazione spesso emergono solo quando il target incontra rete, proxy, credenziali e carichi reali. Una diagnosi efficace parte dai segnali e dal percorso della richiesta, non dalla modifica casuale di configurazioni.

5.15.1 La readiness fallisce, ma il processo è attivo

Separare liveness da readiness e leggere il dettaglio del controllo che fallisce. Verificare che hostname, certificati, DNS, accessi database e timeout siano quelli dell'ambiente target. Un health check che dipende da tutte le API esterne può rendere indisponibile l'intero servizio per una dipendenza non critica.

Confrontare configurazione attiva e sorgente attesa senza stampare segreti. Registrare quali provider di configurazione sono abilitati e validare opzioni all'avvio. Se il servizio si avvia con un endpoint esterno vuoto o un valore di default, correggere la validazione invece di aggiungere retry indefiniti.

5.15.2 Errori 401 o 403 dopo il cutover

401 indica in genere un problema di autenticazione; 403 indica identità riconosciuta ma non autorizzata, anche se il contratto può variare. Verificare emittente (`iss`), destinatario (`aud`), scadenze, chiavi di firma, scarto orario e claim ricevuti. Confrontare il token in modo sicuro senza copiarlo nei ticket o nei log.

Controllare mapping di claim e differenze di casing o nomi tra provider. Un framework sorgente può convertire automaticamente gruppi in ruoli, mentre il target richiede una policy esplicita. Testare un utente rappresentativo per ogni profilo e distinguere permesso mancante da route non autenticata. Non disabilitare authorization per "sbloccare il deploy".

5.15.3 502, 503 e timeout intermittenti

Seguire la richiesta attraverso client, gateway, ingress, Kestrel e dipendenze. Confrontare timeout configurati a ogni livello e verificare se un proxy chiude la

connessione prima dell'applicazione. Correlare trace id e timestamp tra sistemi; senza sincronizzazione oraria i log possono sembrare contraddittori.

Osservare pool connessioni, thread pool, code, DNS e saturazione database. Un retry su un timeout può moltiplicare il carico e peggiorare l'incidente. Ridurre temporaneamente il traffico o disattivare la capability può essere più sicuro che aumentare tutti i timeout.

5.15.4 Differenze di serializzazione

Se i test passano ma un client rifiuta la response, confrontare byte e schema: `Content-Type`, casing, proprietà omesse, enum, formato data, precisione numerica e encoding. Verificare opzioni globali del serializer e attributi sul DTO. Un aggiornamento del serializer o un modello con nullability diversa può cambiare il JSON senza modificare la route.

Riprodurre con fixture minime e un test contrattuale. Non patchare la risposta in middleware per correggere un singolo DTO: la regola deve vivere nel mapping o nella configurazione condivisa documentata.

5.15.5 Mismatch di database dopo la migrazione

Prima di correggere i record, verificare query, filtri, timezone, collations, precisione e ordine. Calcolare differenze per categoria e chiave, usando dati minimizzati. Una differenza può provenire da una query N+1 o da una proprietà null, non dalla copia dati.

Confrontare SQL generato e piani d'esecuzione. Verificare se il vecchio ORM applicava soft-delete, tenant filter o conversioni implicite. Non riparare manualmente il database live prima di sapere se il processo di importazione può riprodurre il problema.

5.15.6 Runbook di incidente

Per un errore in rollout:

1. identificare release, percentuale traffico e capability coinvolta;
2. verificare alert e impatto utente, distinguendo sintomo e causa;
3. applicare la soglia predefinita: fermare incremento o attivare rollback;
4. proteggere dati e fermare eventuali writer duplicati;
5. salvare log e metriche necessari, redigendo segreti e dati personali;
6. assegnare un responsabile tecnico e un comunicatore;
7. dopo la stabilizzazione, documentare causa, rilevazione e prevenzione.

Non fare debug direttamente in produzione con logging verboso permanente o query non controllate. Ogni modifica temporanea ha owner, scadenza e piano di rimozione.

5.16 Verifica finale: criteri di successo

Considerare la migrazione conclusa solo quando il target possiede chiaramente le capability previste e il sistema sorgente non mantiene dipendenze operative nascoste. Account, job, backup, credenziali e supporto devono avere un owner nel nuovo assetto o una data di dismissione.

Misurare il successo con criteri concordati: flussi critici conformi, error rate e latenza entro budget, dati riconciliati, sicurezza approvata, rollback provato, operatori formati e costi del vecchio sistema rimossi o motivati. Conservare le decisioni e i report in modo accessibile al team che sarà on-call.

5.16.1 Fonti ufficiali e aggiornamento

Consultare documentazione ufficiale e controllare la versione target prima di applicare un esempio. Le pagine evolvono; per un progetto reale annotare la data di verifica e la versione .NET:

- .NET — Porting to .NET
- .NET — Panoramica del ciclo di vita
- ASP.NET Core — Panoramica
- ASP.NET Core — Middleware
- ASP.NET Core — Dependency Injection
- ASP.NET Core — Configurazione
- ASP.NET Core — Autenticazione
- ASP.NET Core — Test d'integrazione
- Entity Framework Core — Migrazioni
- Entity Framework Core — Performance
- .NET — Docker
- .NET — Architettura delle applicazioni

5.17 Checklist, glossario e anti-pattern

Una migrazione efficace è una sequenza di decisioni verificabili. Le checklist che seguono non sostituiscono il giudizio del team: aiutano a non dimenticare dipendenze e criteri prima di una fase irreversibile.

5.17.1 Checklist di assessment

- ☐ Sono stati identificati owner funzionali e tecnici per ogni capacità?
- ☐ Sono state inventariate route, job, consumer, webhook e strumenti amministrativi?
- ☐ Sono noti database, writer, volumi, vincoli e consumer dello schema?
- ☐ Sono state elencate integrazioni, librerie, licenze e dipendenze non supportate?
- ☐ Esistono baseline di latenza, throughput, errori, disponibilità e costi?

- ☐ Sono documentati RTO, RPO, retention e requisiti di sicurezza?
- ☐ I comportamenti impliciti e i casi limite sono stati verificati con client reali o test?
- ☐ Le incertezze sono attività con owner, evidenza attesa e scadenza?

5.17.2 Checklist di progettazione

- ☐ La strategia (big bang, componenti, strangler o ibrida) è motivata dal rischio?
- ☐ Ogni unità migrabile ha un confine e un proprietario dei dati?
- ☐ È definito come instradare traffico e come disattivare il nuovo percorso?
- ☐ I contratti API/dati hanno versioni, esempi e test?
- ☐ Il modello di autenticazione e la migrazione delle sessioni sono approvati?
- ☐ Le modifiche schema sono compatibili e il rollback è possibile?
- ☐ Sono stati definiti timeout, retry, idempotenza e cancellazione?
- ☐ Le differenze intenzionali rispetto alla sorgente sono state approvate?

5.17.3 Checklist prima del cutover

- ☐ Build, test, scansioni e pubblicazione passano da checkout pulito.
- ☐ Il target è stato avviato in un ambiente simile alla produzione.
- ☐ Test di caratterizzazione e differenziali non hanno differenze critiche.
- ☐ Sono verificati status, JSON, date, autorizzazioni, paginazione e limiti.
- ☐ Dashboard, alert, log strutturati e correlation id funzionano.
- ☐ Health check e shutdown sono provati con il load balancer.
- ☐ Backup e restore sono stati testati; lo schema è compatibile con rollback.
- ☐ Runbook identifica responsabili, soglie, canali e comandi di ritorno.
- ☐ Il canary ha una durata, percentuale e criterio per proseguire.
- ☐ Le operazioni di scrittura non vengono replicate senza idempotenza.

5.17.4 Checklist per la dismissione

- ☐ Non esiste traffico, job, webhook o client che dipenda ancora dalla sorgente.
- ☐ I dati sono riconciliati e la retention dei backup è approvata.
- ☐ Credenziali, chiavi e route obsolete sono revocate.
- ☐ Le dipendenze residue e la coesistenza hanno owner o data di rimozione.
- ☐ Runbook, supporto e dashboard sono aggiornati sul sistema definitivo.
- ☐ Costi e risorse del sistema precedente sono stati rimossi o motivati.

5.17.5 Glossario essenziale

Termine	Significato nel capitolo
Assessment	Raccolta di evidenze sul sistema, i suoi contratti, vincoli e rischi

Termine	Significato nel capitolo
Baseline	Misure del sistema sorgente usate per confrontare il target
Cutover	Passaggio del traffico o della proprietà di una capacità al target
Strangler	Strategia che sostituisce un sistema per capacità, mantenendo temporaneamente l'originale
Canary	Rilascio progressivo a una parte del traffico prima dell'estensione
Parità funzionale	Equivalenza del comportamento osservabile per i flussi concordati
Idempotenza	Proprietà per cui ripetere un'operazione non duplica l'effetto
Outbox	Registro transazionale degli eventi da consegnare dopo il commit
RTO/RPO	Obiettivo di tempo di ripristino e perdita dati massima accettabile
Expand–migrate–contract	Evoluzione graduale di schema e codice mantenendo compatibilità temporanea

5.17.6 Gli errori da evitare

1. **Riscrivere senza caratterizzare.** Il team si affida a ricordi e documenti incompleti; i test sul comportamento esistente vanno creati prima del cutover.
2. **Confondere parità e miglioramento.** Cambiare contratto, dati e architettura nello stesso rilascio rende impossibile isolare le cause delle regressioni.
3. **Tradurre i nomi invece delle responsabilità.** Un middleware o un service container con nome simile può avere lifecycle diverso; verificare semantica e scope.
4. **Ignorare il database condiviso.** Le tabelle possono essere un contratto di altri servizi; identificare tutti i writer e reader prima di migrare.
5. **Duplicare scritture senza protocollo.** Due sistemi non sono sincronizzati perché entrambi hanno ricevuto una chiamata; progettare idempotenza e riconciliazione.
6. **Confondere rollback applicativo e rollback dati.** La versione precedente può non interpretare i dati scritti dalla nuova.
7. **Ottimizzare prima di misurare.** AOT, cache o parallelismo aumentano complessità se il collo di bottiglia è altrove.
8. **Considerare test unitari come prova di produzione.** Database, proxy, identity e container richiedono test d'integrazione mirati.

9. **Lasciare la coesistenza senza data di uscita.** Il doppio sistema raddoppia costi e superficie di rischio; assegnare owner e milestone di dismissione.
10. **Fare del cutover una scommessa.** Soglie, responsabilità e rollback devono essere definiti e provati prima che il traffico venga spostato.

5.18 Appendice: piano di migrazione riutilizzabile

Il seguente schema può essere copiato nel backlog di un progetto e adattato. Ogni riga deve avere owner, data, evidenza e stato; non lasciare caselle spuntate sulla sola base di una dichiarazione.

Fase	Deliverable	Criterio di completamento
Discovery	Inventario, baseline, rischi e contratti	Owner e dipendenze identificati; lacune tracciate
Foundation	Repo, SDK, CI, ambiente, logging	Build pulita, deploy e health check provati
Caratterizzazione	Fixture e test sorgente	Flussi critici riproducibili e approvati
Vertical slice	Endpoint target completo	HTTP, dominio, storage e test passano
Parità	Confronto sorgente/target	Differenze critiche risolte o approvate
Canary	Traffico controllato	Soglie rispettate per la finestra concordata
Cutover	Ownership e routing aggiornati	Target proprietario; rollback ancora possibile
Stabilizzazione	Runbook e supporto	Incidenti e metriche entro obiettivi
Dismissione	Sistema sorgente spento	Traffico nullo, accessi revocati, dati conservati

Per ogni capacità, annotare un punteggio qualitativo di rischio (basso/medio/alto) e le evidenze: non sommare numeri senza significato. Se il criterio non è raggiunto, la decisione corretta può essere fermarsi e ridurre l'incertezza, non accelerare il rilascio.